INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] DevOps and Security

LS lab 3 Report

On

CI/CD Infrastructure

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 09.02.2026

Index

Abstract…….…. **3**

Task 1: Infra Deployment…… **4-29**

Task 2: Create CI/CD Pipeline…….. **30-43**

Task 3: Polish the CICD……. **43-54**

Task 4: Backup and Disaster Recovery Scenario (Bonus)…….. **54-63**

References……... **63-64**

Abstract:

In this work, the objective of LS Lab 3 (CI/CD Infrastructure) is to design, implement and validate a complete CI/CD system in a real-world environment. The entire process from start to finish is designed in such a way that it is not just a lab task but a small but complete version of a real-life DevOps workflow. A responsibility-based infrastructure is created using three separate Ubuntu VMs - a GitLab Server, a GitLab Runner and a Deployment Server, so that production-grade separation of concerns is maintained. GitLab is deployed on the GitLab Server using Docker Compose and local DNS mapping and network connectivity are verified to ensure domain-based access. GitLab Runner is installed in the Runner VM in a non-docker manner and the shell executor is configured, so that CI jobs can run directly on the operating system and Docker build/push is completed realistically. A completely automation-friendly environment is created by setting up a separate deploy user, SSH key-based authentication, Docker permission and passwordless *sudo* on the Deployment Server, so that the CI/CD pipeline can run the deployment without any manual intervention.

Then, a simple but meaningful web app with Frontend and Backend has been created, where Flask API is used as Backend and static HTML/JavaScript is used as Frontend. To verify this app in a test-driven manner, automated tests have been added using *pytest* and everything has been packaged together in a Docker image. Both CI and CD steps have been implemented together using a single *.gitlab-ci.yml* file - build, test, Docker image build and push to Docker Hub, and finally deploy to Deployment Server using Ansible. To maintain security, all secret information has been managed using GitLab CI/CD Variables and cache, artifacts, needs and dependencies have been consciously implemented to increase the speed and control of the pipeline. Different behavior has been ensured for develop and master branches by implementing multi-branch workflow, where CI is running on the develop branch and full CD is implemented on the master branch. To prove the deployment was successful, server-side (curl), browser-level (UI interaction) and container-level verifications were performed, as well as the traceability of the deployed version using commit *SHA*. Finally, repository and file recovery were verified at both the filesystem and Web UI levels by taking a hot backup of GitLab, intentionally destroying data, and restoring it completely from that backup.

Overall, this work demonstrates an end-to-end CI/CD and disaster recovery workflow, which clearly reflects the core concepts of DevOps: automation, reliability, security and traceability.

Task 1: Infra Deployment

1. Deploy three VMs that you will be using as Gitlab Server, Gitlab Runner, and the deployment server. Make sure VMs can reach each other.

Solution:

STX=24

IP plan:

- VM1: 10.10.10.11

```
hasib@gitlab-24:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ac:5e:e4 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 10.10.10.11/24 brd 10.10.10.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feac:5ee4/64 scope link
       valid_lft forever preferred_lft forever
```

- VM2: 10.10.10.12

```
hasib@runner-24:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:90:03:5a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 10.10.10.12/24 brd 10.10.10.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe90:35a/64 scope link
       valid_lft forever preferred_lft forever
```

- VM3: 10.10.10.13

```
deploy@deploy-24:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:31:8f:53 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 10.10.10.13/24 brd 10.10.10.255 scope global ens33
       valid_lft forever preferred_lft forever
```

Three VM deployments + Connectivity:

Basic update on each VM:

VM1

```
hasib@gitlab-24:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 https://download.docker.com/linux/ubuntu noble InRelease
Hit:6 https://packages.gitlab.com/gitlab/gitlab-ce/ubuntu noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
The following upgrades have been deferred due to phasing:
  language-pack-gnome-en language-pack-gnome-en-base
The following packages will be upgraded:
  alsa-ucm-conf language-pack-en language-pack-en-base linux-libc-dev linux-tools-common
5 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
Need to get 3,438 kB of archives.
After this operation, 256 kB of additional disk space will be used.
Get:1 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 language-pack-en all 1:24.04+20260127 [1,892 B]
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 language-pack-en-base all 1:24.04+20260127 [446 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 alsa-ucm-conf all 1.2.10-1ubuntu5.9 [67.8 kB]
Get:4 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-libc-dev amd64 6.8.0-100.100 [2,042 kB]
```

VM2

```
hasib@runner-24:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:5 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease
Get:6 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,739 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,528 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:12 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,312 B]
Get:13 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:15 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:16 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Fetched 4,321 kB in 9s (483 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
```

VM3

```
deploy@deploy-24:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,739 kB]
Get:6 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,528 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:11 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,312 B]
Get:12 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:13 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:15 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Fetched 4,321 kB in 8s (517 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

apt update was run to refresh the system package list to get the latest package information from the repository. Then, the installed packages were updated using apt upgrade *-y* , where *-y* did the auto-confirmation. These steps were done before setting up Docker, Git, and Runner in the CI/CD lab to reduce dependency issues. The output showed Hit: or Get: to indicate that the repository worked properly, and at the end, 0 upgraded or X upgraded are both normal and mean the update or install was successful.

VM1

```
hasib@gitlab-24:~$ sudo apt install -y nano curl wget git ca-certificates gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nano is already the newest version (7.2-2ubuntu0.1).
curl is already the newest version (8.5.0-2ubuntu10.6).
wget is already the newest version (1.21.4-1ubuntu4.1).
git is already the newest version (1:2.43.0-1ubuntu7.3).
ca-certificates is already the newest version (20240203).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
lsb-release is already the newest version (12.0-2).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
```

VM2

```
hasib@runner-24:~$ sudo apt install -y nano curl wget git ca-certificates gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nano is already the newest version (7.2-2ubuntu0.1).
curl is already the newest version (8.5.0-2ubuntu10.6).
wget is already the newest version (1.21.4-1ubuntu4.1).
git is already the newest version (1:2.43.0-1ubuntu7.3).
ca-certificates is already the newest version (20240203).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
lsb-release is already the newest version (12.0-2).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

VM3

```
deploy@deploy-24:~$ sudo apt install -y nano curl wget git ca-certificates gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nano is already the newest version (7.2-2ubuntu0.1).
curl is already the newest version (8.5.0-2ubuntu10.6).
wget is already the newest version (1.21.4-1ubuntu4.1).
git is already the newest version (1:2.43.0-1ubuntu7.3).
ca-certificates is already the newest version (20240203).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
lsb-release is already the newest version (12.0-2).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

nano is installed for easy editing of configuration files. *curl* and *wget* are used for downloading repository keys, fetching data from URLs, and making API calls. *git* is also needed for cloning, pushing, and pulling repositories. ca-certificates ensures *HTTPS* certificate trust, which is needed when fetching packages from Docker and GitLab repositories. *gnupg* is used to add and verify GPG keys, especially in Docker and GitLab Runner repository setup. *lsb-release* helps identify the codename and version of the distro, making it easier to configure the correct repository.

Setting Hostname:

To ensure the distinct identity of the VMs, a hostname has been set for each VM. VM1 has been given the *hostname gitlab-24*, VM2 has been given the *hostname runner-24*, and VM3 has been given the *hostname deploy-24*, so that it is clear which VM is the GitLab server, which is the Runner, and which is the Deploy server.

VM1

```
hasib@gitlab-24:~$ sudo hostnamectl set-hostname gitlab-24
```

VM2

```
hasib@runner-24:~$ sudo hostnamectl set-hostname runner-24
hasib@runner-24:~$
```

VM3

```
deploy@deploy-24:~$ sudo hostnamectl set-hostname deploy-24
```

To verify that the hostname has been set, run the *hostname* or *hostnamectl* command. The output shows *gitlab-24* on VM1, *runner-24* on VM2, and *deploy-24* on VM3, indicating that the hostname has been set successfully.

/etc/hosts updated to meet DNS requirement:

The /etc/hosts file has been updated in all VMs to create local DNS mapping. Since there is no real DNS server in this CI/CD lab, the hostname - IP resolution is done through the /etc/hosts file. As a result, each VM is able to recognize other VMs by hostname or domain name. Since as per the lab requirement, the GitLab server has to be exposed as *24.sne.com*, adding *10.10.10.11 24.sne.com gitlab-24* to */etc/hosts* means that from where *24.sne.com* is typed, it is directly resolved to the IP of VM1 (GitLab server), and the name *gitlab-24* also works as an alias on the same IP.

The following configuration has been added to all VMs through the following command.

VM1

```
hasib@gitlab-24:~$ sudo nano /etc/hosts
```

VM2

```
hasib@runner-24:~$ sudo nano /etc/hosts
```

VM3

```
deploy@deploy-24:~$ sudo nano /etc/hosts
```

The following lines have been added to the configuration file in the Nano editor:

VM1

```
  GNU nano 7.2                                                    /etc/hosts
127.0.0.1   localhost
127.0.1.1   gitlab-24

# GitLab CI/CD Lab Hosts
10.10.10.11  24.sne.com gitlab-24
10.10.10.12  runner-24
10.10.10.13  deploy-24
```

VM2

```
  GNU nano 7.2
127.0.0.1   localhost
127.0.1.1   gitlab-24

# GitLab CI/CD Lab Hosts
10.10.10.11  24.sne.com gitlab-24
10.10.10.12  runner-24
10.10.10.13  deploy-24
```

VM3

```
  GNU nano 7.2                                                    /e
127.0.0.1   localhost
127.0.1.1   gitlab-24

# GitLab CI/CD Lab Hosts
10.10.10.11  24.sne.com gitlab-24
10.10.10.12  runner-24
10.10.10.13  deploy-24
```

127.0.0.1 localhost is the default loopback entry. *127.0.1.1 gitlab-24* is kept as the local hostname mapping on many *Ubuntus*. This is harmless. Adding 10.10.10.12 runner-24 and *10.10.10.13 deploy-24* resolves the hostnames of the runner and deploy servers correctly. As a result of this setup, the GitLab server can be accessed directly from VM2 and VM3 using *http://24.sne.com*.

Ping test (VM reachability):

A ping test was performed to prove that the network connectivity and hostname resolution between the VMs are correct. The reachability of the GitLab server was checked by sending an ICMP

request by running the following command from VM2, where -c 3 was used to send only 3 packets, not infinitely.

```
hasib@runner-24:~$ ping -c 3 24.sne.com
```

The output below shows *ping 24.sne.com (10.10.10.11)* which proves that *24.sne.com* is correctly resolved to *10.10.10.11* (VM1) according to */etc/hosts. 64 bytes from ... time=... ms* indicates that the network from VM2 to VM1 is working properly. *0% packet loss* means connectivity is stable, no packets are being dropped, and *rtt min/avg/max/mdev* are just informational stats for latency.

```
hasib@runner-24:~$ ping -c 3 24.sne.com
PING 24.sne.com (10.10.10.11) 56(84) bytes of data.
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=1 ttl=64 time=3.06 ms
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=2 ttl=64 time=1.64 ms
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=3 ttl=64 time=9.69 ms

--- 24.sne.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.640/4.795/9.689/3.508 ms
```

Then the following command is run from VM2. Here PING deploy-24 (*10.10.10.13*) shows that hostname resolve is *OK* and *0% packet loss* means that network connectivity from VM2 to VM3 is OK.

```
hasib@runner-24:~$ ping -c 3 deploy-24
```

```
hasib@runner-24:~$ ping -c 3 deploy-24
PING deploy-24 (10.10.10.13) 56(84) bytes of data.
64 bytes from deploy-24 (10.10.10.13): icmp_seq=1 ttl=64 time=7.17 ms
64 bytes from deploy-24 (10.10.10.13): icmp_seq=2 ttl=64 time=1.17 ms
64 bytes from deploy-24 (10.10.10.13): icmp_seq=3 ttl=64 time=1.89 ms

--- deploy-24 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.172/3.412/7.174/2.676 ms
```

The VM3 to VM1 connectivity has been validated by running the following command from VM3.

```
deploy@deploy-24:~$ ping -c 3 24.sne.com
```

```
deploy@deploy-24:~$ ping -c 3 24.sne.com
PING 24.sne.com (10.10.10.11) 56(84) bytes of data.
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=1 ttl=64 time=5.05 ms
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=2 ttl=64 time=1.76 ms
64 bytes from 24.sne.com (10.10.10.11): icmp_seq=3 ttl=64 time=1.27 ms

--- 24.sne.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.273/2.693/5.053/1.679 ms
```

Ping ok is found in the above output.

2. Set up Gitlab Server (VM1), and create a docker-compose file with the below configs:

a. Pull the Gitlab EE or CE edition

b. Name the running container as <stx>-gitlab

c. Map container ports 80 and 22 to host machine

d. Expose the Gitlab server as <stx>.sne.com. (Hint: find the right env variable to pass to the container to update the configs of Gitlab. Update the *hosts* file to resolve the mentioned DNS record)

e. Bind the necessary directories of the Gitlab server container to the host machine (e.g. logs, app data, configs…)

f. Run the docker-compose file and make sure the configs are working.

g. Access the Gitlab server and log in, create a project name it as <stx>-repo.

h. etc…

Note: you can check https://docs.gitlab.com/ee/install/docker.html

Solution:

Setting up Gitlab Server (VM1), and creating a docker-compose file:

Docker and Compose install:

The following commands were run to install Docker and Docker Compose on Ubuntu. Here, the *docker.io* package installs the Docker engine from Ubuntu's official repository, which is the main service for running containers. The *docker-compose-plugin* is used to run the new Compose v2, where the command is docker compose. The *-y* option is given so that the installation is completed automatically without asking for any confirmation.

```
hasib@gitlab-24:~$ sudo apt install -y docker.io docker-compose-plugin
```

The output shows *docker-compose-plugin* is already the newest version, which means that this package was already installed on the system and is the latest version. This is not an error, but rather proof that Docker Compose is properly present and does not need to be reinstalled.

```
hasib@gitlab-24:~$ sudo apt install -y docker.io docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-compose-plugin is already the newest version (5.0.2-1~ubuntu.24.04~noble)
.
Some packages could not be installed. This may mean that you have
requested an impossible situation or if you are using the unstable
distribution that some required packages have not yet been created
or been moved out of Incoming.
The following information may help to resolve the situation:

The following packages have unmet dependencies:
 containerd.io : Conflicts: containerd
```

The following command is used to start and permanently activate the Docker service. Here, the enable option ensures that Docker will start automatically as soon as the system boots, and the --*now* option starts the Docker service immediately.

```
hasib@gitlab-24:~$ sudo usermod -aG docker $USER
```

I have seen the output below as *Synchronizing state...*, it means that the Docker service has been successfully enabled and configured for boot.

```
hasib@gitlab-24:~$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/sys
temd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
```

The following steps are used to run Docker as a non-root user, so that you don't have to use sudo repeatedly. The following command adds the current user to the docker group, giving that user permission to work directly with the Docker daemon.

```
hasib@gitlab-24:~$ sudo usermod -aG docker $USER
```

Then the following command is run so that the new group membership is applied immediately without having to logout or reboot.

```
hasib@gitlab-24:~$ newgrp docker
```

A version check was performed to verify that Docker and Docker Compose were installed correctly. The following command confirmed the installation and version of the Docker engine, and the following command verified that the Compose plugin was working properly.

```
hasib@gitlab-24:~$ docker --version
Docker version 29.2.1, build a5c7197
```

The output shows *Docker version 29.2.1...,* indicating that the Docker engine was successfully installed. *Docker Compose version v5.0.2* confirms that the Docker Compose plugin is installed and ready to use the docker compose command.

```
hasib@gitlab-24:~$ docker compose version
Docker Compose version v5.0.2
```

Bind directories create:

A bind directory has been created on the host to store the persistent data of the GitLab container. The following command has created three directories at once.

```
hasib@gitlab-24:~$ sudo mkdir -p /srv/gitlab-24/{config,logs,data}
```

The ownership of these has been given to the current user with the following command so that there are no permission issues.

```
hasib@gitlab-24:~$ sudo chown -R $USER:$USER /srv/gitlab-24
```

Here /srv/gitlab-24*/config* will be used to hold GitLab's */etc/gitlab* configuration files, /srv/gitlab-24*/logs* will map to /var/log/gitlab logs, and /srv/gitlab-*24/data* will map to */var/opt/gitlab,* which contains repositories, databases, uploads, etc. Binding these directories ensures that even if the container is deleted or recreated, the original data remains safe on the host, which is mandatory for completing the backup/restore bonus task.

Creating docker-compose.yml:

To create the Docker Compose configuration for GitLab, a dedicated project folder *(~/gitlab-24*) was first created, so that the Compose file is in one place and easy to edit, backup, or restore in the future.

```
hasib@gitlab-24:~$ mkdir -p ~/gitlab-24
```

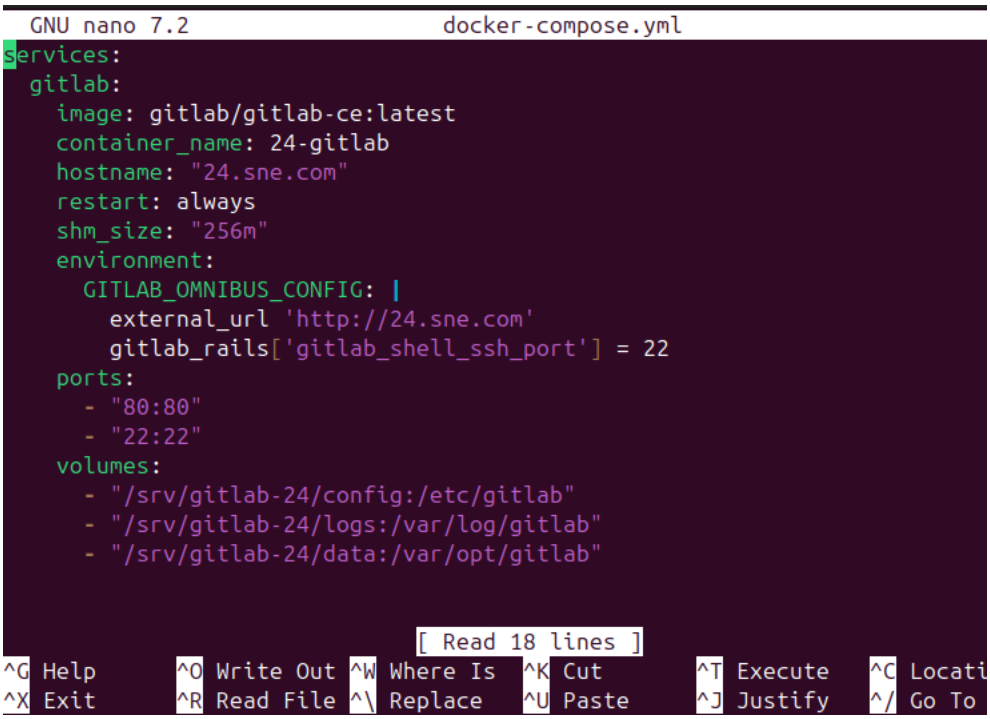
```
hasib@gitlab-24:~$ cd ~/gitlab-24
```

The file below defines a single service (*gitlab*), using the *gitlab/gitlab-ce:latest* image, which is lightweight and acceptable for the lab as it is Community Edition. The container name is *24-gitlab* as per the lab requirement, and the hostname: *24.sne.com* is set so that the internal config and URL of GitLab are the same.

```
hasib@gitlab-24:~/gitlab-24$ nano docker-compose.yml
```

By setting restart: always, GitLab restarts itself even if the VM reboots or the container crashes. *shm_size: 256m* is used to keep GitLab's internal services stable. Inline Omnibus config is provided

through the *GITLAB_OMNIBUS_CONFIG* environment variable, where external_url is set to *'http://24.sne.com'*, this domain is used in GitLab UI, clone URL and pipeline link, and SSH clone port is declared with *gitlab_shell_ssh_port = 22*. Port mapping is used to ensure direct GitLab web and SSH access from the host using *80:80* and *22:22*. */srv/gitlab-24/config*, */logs* and */data* are bound in the volumes section, so that GitLab's config, logs and actual data (repos, DB, uploads) are saved on the host outside the container.



Running GitLab container:

The following command is used to start the GitLab container. Here, the up service is created and started, and the -d option is given to run the container in the background (detached mode), so the terminal is not blocked.

```
hasib@gitlab-24:~/gitlab-24$ docker compose up -d
```

Pulling ... 1.768GB is shown in the output, it means that the GitLab image has been downloaded. Because the GitLab image is large in size. *Network ... Created* is shown, it means that Docker Compose has created a separate network for this stack. And *Container 24-gitlab* Created means that the container has been created successfully.

```
hasib@gitlab-24:~/gitlab-24$ docker compose up -d
[+] up 8/10
[+] up 8/10tlab/gitlab-ce:latest [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 1.768GB / 1.768GB Pulling    191.5s
[+] up 12/12lab/gitlab-ce:latest [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 1.768GB / 1.768GB Pulling    245.5s
 ✔ Image gitlab/gitlab-ce:latest Pulled                                          260.1s
 ✔ Network gitlab-24_default     Created                                         0.2s
 ✔ Container 24-gitlab           Created                                         2.2s
```

The container has been verified with the following command. If the output shows *STATUS ... (health: starting)*, that is also expected, because GitLab does the initial setup in the first few minutes.

```
hasib@gitlab-24:~/gitlab-24$ docker ps
```

The status below shows healthy, indicating that GitLab is fully ready.

```
hasib@gitlab-24:~/gitlab-24$ docker ps
CONTAINER ID   IMAGE                         COMMAND                  CREATED
   STATUS                             PORTS
                                          NAMES
17d61b59617b   gitlab/gitlab-ce:latest   "/assets/init-contai…"   15 seconds ago
   Up 13 seconds (health: starting)   0.0.0.0:22->22/tcp, [::]:22->22/tcp, 0.0.0
.0:80->80/tcp, [::]:80->80/tcp, 443/tcp   24-gitlab
```

Retrieving the root password:

The following command is used to retrieve the initial root password that is generated when the GitLab container is first started. Here, the command is executed inside the running container using *docker exec -it 24-gitlab*, the command is executed as a login shell using bash *-lc*, and the initial root password is read from the */etc/gitlab/initial_root_password* file.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab bash -lc "cat /etc/gitlab/initial_root_password"
```

A Warning appears in the output. The value of the *Password: ...* line below the Warning is the original initial root password. GitLab usually deletes this file automatically after about 24 hours.

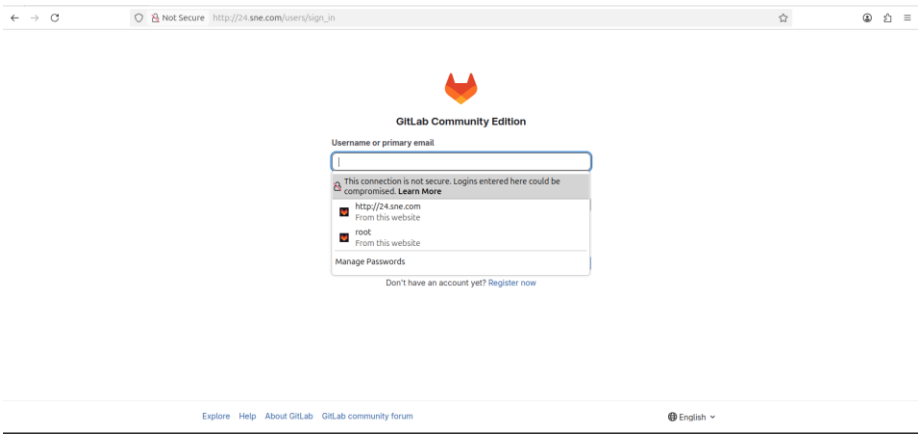
```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab bash -lc "cat /etc/gitlab/initial_root_password"
# WARNING: This password is only valid if ALL of the following are true:
#          • You set it manually via the GITLAB_ROOT_PASSWORD environment variable
#            OR the gitlab_rails['initial_root_password'] setting in /etc/gitlab/gitlab.rb
#          • You set it BEFORE the initial database setup (typically during first installation)
#          • You have NOT changed the password since then (via web UI or command line)
#
#          If this password doesn't work, reset the admin password using:
#          https://docs.gitlab.com/security/reset_user_password/#reset-the-root-password

Password: +EkKoixtNSzq98T81LjPtyNflRRpkWCHyo0WhVbwJac=

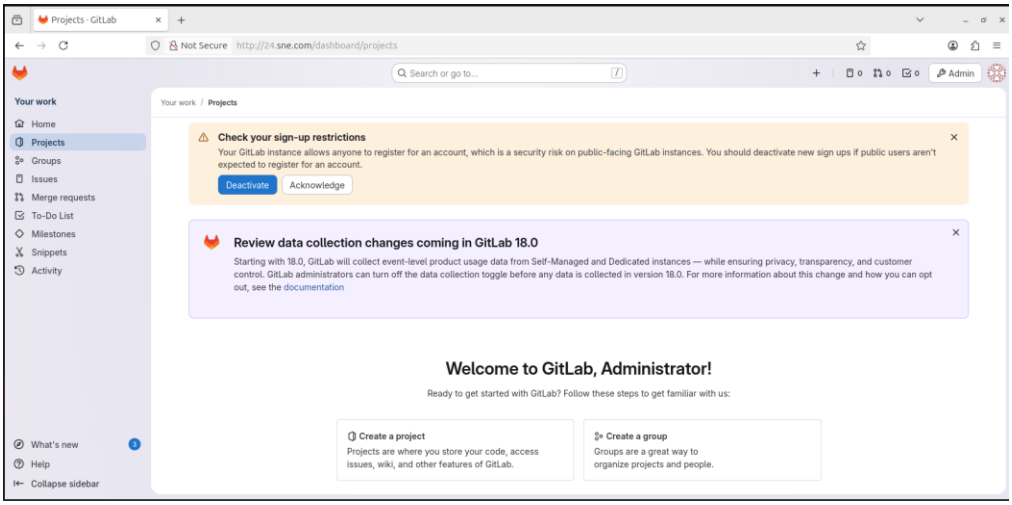
# NOTE: This file is automatically deleted after 24 hours on the next reconfigure run.
```

GitLab UI-Project created 24-repo:

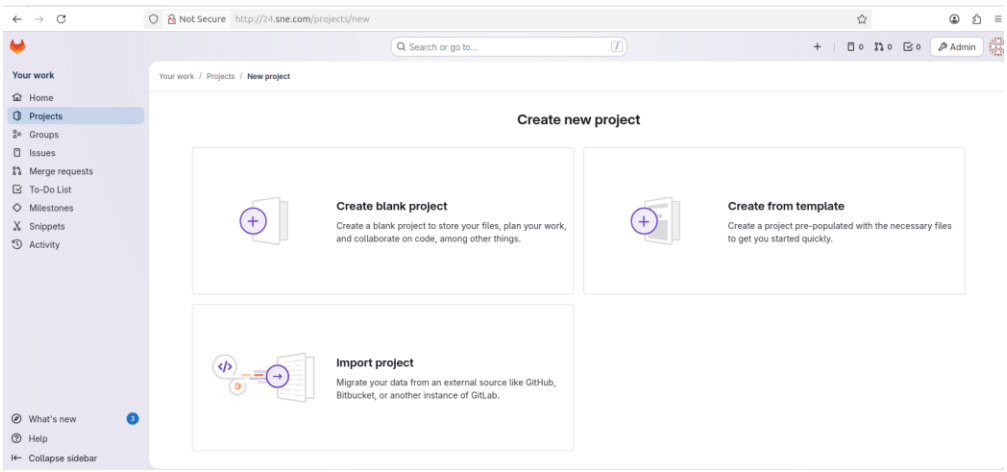
Opened *http://24.sne.com* from browser. As previously mapped in */etc/hosts*, the name *24.sne.com* resolved correctly to *10.10.10.11* (*GitLab VM*) and GitLab UI could be accessed.



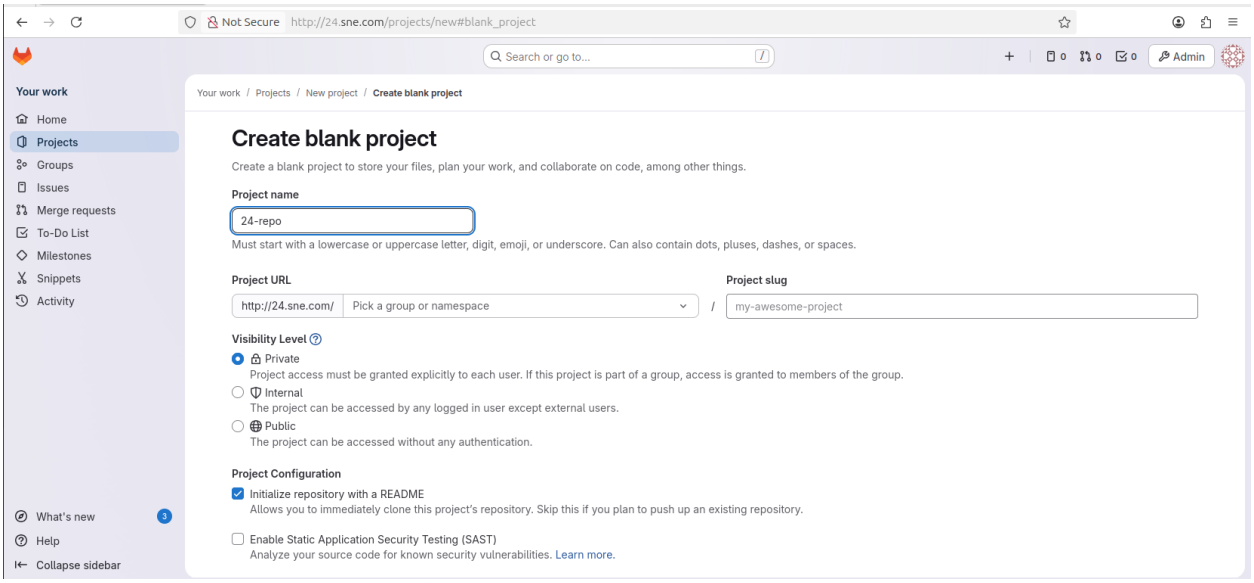
Then, login was done using the root username and *initial_root_password* obtained from the container.



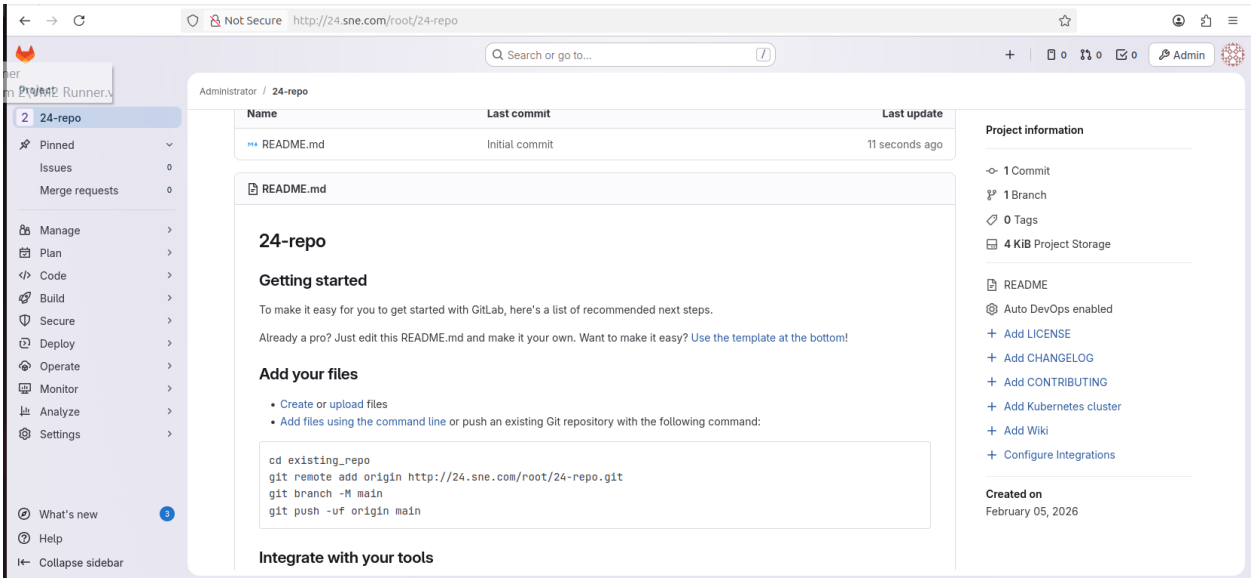
After logging in, go to Projects from the left sidebar and select *Create blank project* from *New project*.



The project name is given as *24-repo*.



Then, I clicked *Create project.*



3. Set up the Gitlab Runner (VM2), don't use the docker approach this time. a. Install and configure shared Gitlab Runner

b. Explain what is the Gitlab runner executor and set the executor type to *shell*

c. Set Gitlab Runner tag to <stx>-runner. (You will be using this tag in the pipeline in the coming task)

d. Authenticate your Gitlab runner with Gitlab server, and validate.

Note: you can also check https://docs.gitlab.com/runner/install/

Solution:

Installing GitLab Runner:

The following command was run to setup the official APT repository before installing GitLab Runner on VM2. This one-line script automatically detected the OS, added the correct APT repository for GitLab Runner, and imported the GPG key of that repository to verify the authenticity of the package.

```
hasib@runner-24:~$ curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
```

The output shows Detected operating system as Ubuntu. This means that the OS has been correctly detected and the repo is set for the correct distro. Installing */etc/apt/sources.list.d/runner_gitlab-runner.list...done*. This means that a new APT repo list file has been created. Importing *packagecloud gpg key...* imported to */etc/apt/keyrings/...gpg* means that the GPG key of the repo

has been successfully added, so trusted installation is possible. Finally, the repository was setup! I installed packages. This confirms that the GitLab Runner repository is fully setup.

```
hasib@runner-24:~$ curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
[sudo] password for hasib:   % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  7228  100  7228    0     0  10481      0 --:--:-- --:--:-- --:--:-- 10475

Sorry, try again.
[sudo] password for hasib:
Detected operating system as Ubuntu/noble.
Checking for curl...
Detected curl...
Checking for gpg...
Detected gpg...
Detected apt version as 2.8.3
Running apt-get update... done.
Installing apt-transport-https... done.
Installing /etc/apt/sources.list.d/runner_gitlab-runner.list...done.
Importing packagecloud gpg key... Packagecloud gpg key imported to /etc/apt/keyrings/runner_gitlab-runner-archive-keyring.gpg
done.
Running apt-get update... done.

The repository is setup! You can now install packages.
```

APT update – applying repo changes:

After adding the repo, an update was required to fetch the package list from the new repo.

```
hasib@runner-24:~$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
```

The following command was run on VM2 to install GitLab Runner.

```
hasib@runner-24:~$ sudo apt install -y gitlab-runner
```

The output shows Need to get *573 MB* ... After this operation, *651 MB* ... because the GitLab Runner and helper images are large in size, so the download and disk usage were expected to be high. Setting up *gitlab-runner* ... creating *gitlab-runner...* means that a system user named *gitlab-runner* has been created in the system, through which runner jobs are executed at the system-level.

```
hasib@runner-24:~$ sudo apt install -y gitlab-runner
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gitlab-runner-helper-images
Suggested packages:
  docker-engine
The following NEW packages will be installed:
  gitlab-runner gitlab-runner-helper-images
0 upgraded, 2 newly installed, 0 to remove and 1 not upgraded.
Need to get 573 MB of archives.
After this operation, 651 MB of additional disk space will be used.
Get:1 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble/main all gitlab-runner-helper-images all 18.8.0-1 [545 MB]
Get:2 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble/main amd64 gitlab-runner amd64 18.8.0-1 [27.4 MB]
Fetched 573 MB in 32s (17.9 MB/s)
Selecting previously unselected package gitlab-runner-helper-images.
(Reading database ... 152799 files and directories currently installed.)
Preparing to unpack .../gitlab-runner-helper-images_18.8.0-1_all.deb ...
Unpacking gitlab-runner-helper-images (18.8.0-1) ...


Selecting previously unselected package gitlab-runner.
Preparing to unpack .../gitlab-runner_18.8.0-1_amd64.deb ...
Unpacking gitlab-runner (18.8.0-1) ...
Setting up gitlab-runner-helper-images (18.8.0-1) ...
Setting up gitlab-runner (18.8.0-1) ...
```

Enable & start now:

The following command is used to start and permanently activate the GitLab Runner service. The enable option here ensures that GitLab Runner starts automatically after the system boots, and the *--now* option ensures that the service starts immediately.

```
hasib@runner-24:~$ sudo systemctl enable --now gitlab-runner
```

Status check:

The following command was run to verify that GitLab Runner is running properly.

```
hasib@runner-24:~$ sudo systemctl status gitlab-runner --no-pager
```

The output shows *Loaded: loaded ...* enabled which indicates that the service file was loaded correctly and enabled for boot-time auto-start. The *Active: active (running)* line indicates that the GitLab Runner daemon is currently running. The Main *PID: ... (gitlab-runner)* line confirms that the runner process is alive and running under *systemd*. The log section shows Running in system-mode. This means that the runner is running in system mode, i.e. it is acting as a global runner.

```
hasib@runner-24: $ sudo systemctl status gitlab-runner --no-pager
● gitlab-runner.service - GitLab Runner
     Loaded: loaded (/etc/systemd/system/gitlab-runner.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-02-05 21:35:03 MSK; 32s ago
   Main PID: 4751 (gitlab-runner)
      Tasks: 7 (limit: 4544)
     Memory: 23.9M (peak: 24.2M)
        CPU: 393ms
     CGroup: /system.slice/gitlab-runner.service
             └─4751 /usr/bin/gitlab-runner run --config /etc/gitlab-runner/config.toml --working-directory /home/gitlab-runner --ser…

Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Runtime platform                                    arch=amd64 os=linux pid=4…n=18.8.0
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Starting multi-runner from /etc/gitlab-runner/config.toml...  builds=0 max_builds=0
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Running in system-mode.
Feb 05 21:35:04 runner-24 gitlab-runner[4751]:
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Created missing unique system ID                    system_id=s_b020b4202a6a
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Usage logger disabled                               builds=0 max_builds=1
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Configuration loaded                                builds=0 max_builds=1
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: listen_address not defined, metrics & debug endpoints disabled  builds=0 max_builds=1
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: [session_server].listen_address not defined, session endpoints disabled  buil…builds=1
Feb 05 21:35:04 runner-24 gitlab-runner[4751]: Initializing executor providers                     builds=0 max_builds=1
Hint: Some lines were ellipsized, use -l to show in full.
```

Verifying GitLab Runner version:

The following command was run to verify whether GitLab Runner was installed correctly and which version was running.

```
hasib@runner-24:~$ gitlab-runner --version
```

The *Version: 18.8.0* in the output indicates the installed version of GitLab Runner.

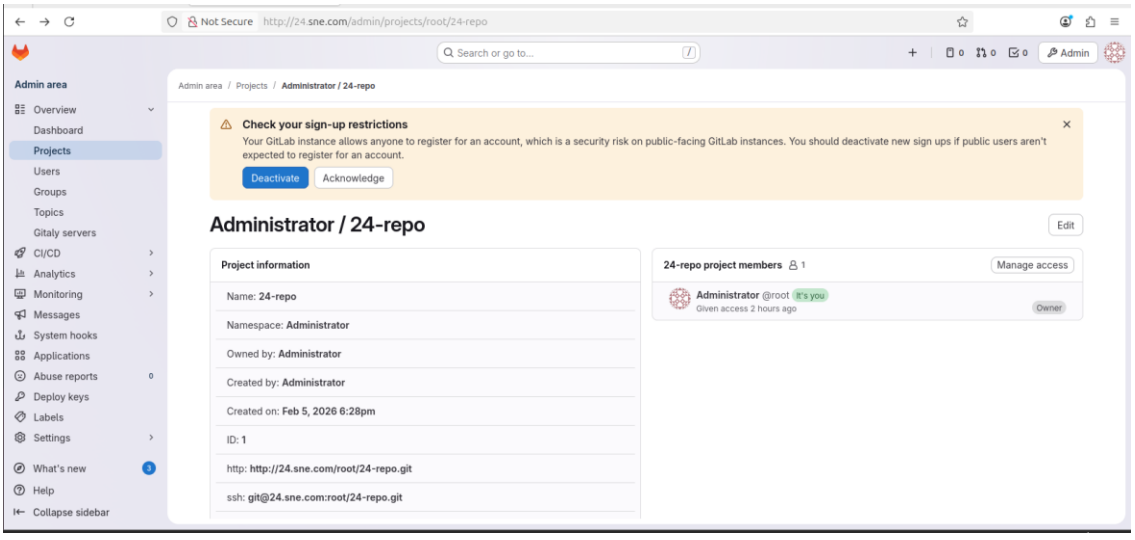
```
hasib@runner-24:~$ gitlab-runner --version
Version:      18.8.0
Git revision: 9ffb4aa0
Git branch:   18-8-stable
GO version:   go1.25.3 X:cacheprog
Built:        2026-01-15T15:55:25Z
OS/Arch:      linux/amd64
```

Runner executor explanation:

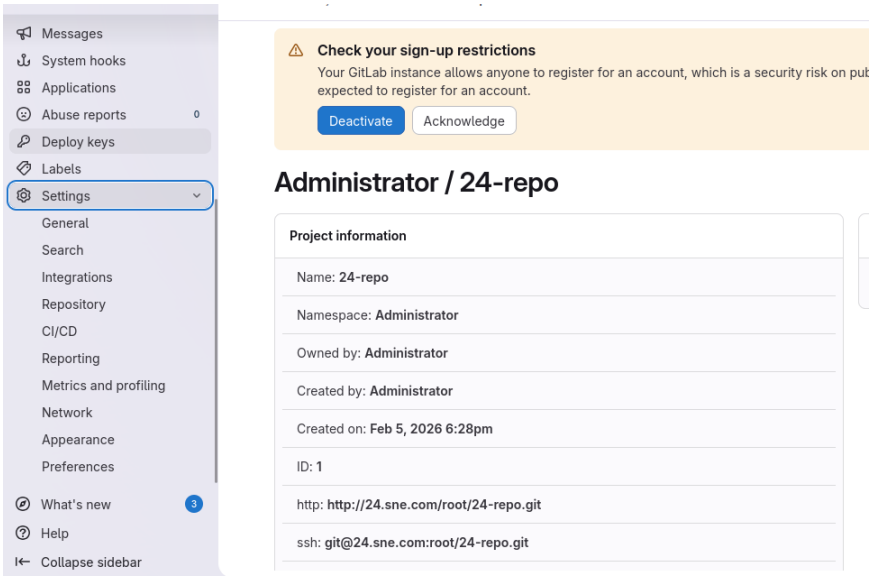
In Shell executor, CI jobs are executed directly on the local operating system of Runner VM (VM2) using bash shell. Shell executor executes CI jobs in local bash shell of Runner VM.

Getting Runner token from GitLab UI:

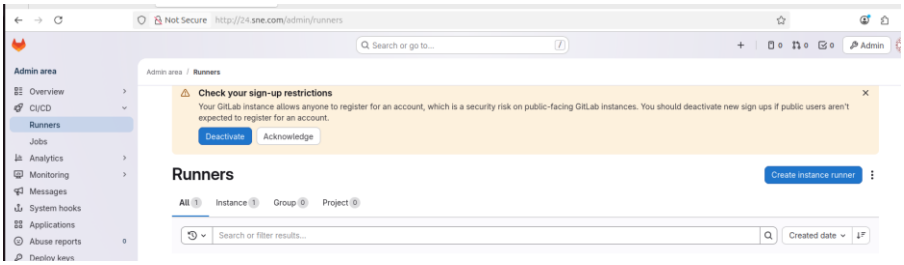
In this step, Runner registration token is collected from GitLab UI, which is used to authenticate GitLab Runner of VM2 with specific project (24-repo). Opening *http://24.sne.com* from browser, which is resolved to GitLab server (VM1) through */etc/hosts* mapping.



After logging in, 24-repo is opened from the project list, because the runner executes CI jobs for this specific project.



Then, from the left sidebar, I have gone to Settings, CI/CD, and expand the Runners section. Here, I have seen the runner-related status, token, and instructions.



At the end of this step, the GitLab instance URL (*http://24.sne.com*) and project-level runner registration token are obtained.

Runner register:

GitLab Runner Repository & Package (Re-confirmation):

The following command was run again to re-confirm that the GitLab Runner repository and package setup are correct. This script automatically detects my OS, adds the official APT repository for GitLab Runner, imports the corresponding GPG key, and finally runs apt update so that the system recognizes the packages from the new repository.

```
hasib@runner-24:~$ curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
```

The output shows Detected operating system as Ubuntu/noble, which means the OS has been detected correctly. And the output shows *The repository is setup*!, which confirms that the GitLab Runner repository has been successfully added.

```
hasib@runner-24:~$ curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
[sudo] password for hasib:   % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  7228  100  7228    0     0   6245      0  0:00:01  0:00:01 --:--:--  6252

Sorry, try again.
[sudo] password for hasib:
Detected operating system as Ubuntu/noble.
Checking for curl...
Detected curl...
Checking for gpg...
Detected gpg...
Detected apt version as 2.8.3
Running apt-get update... done.
Installing apt-transport-https... done.
Installing /etc/apt/sources.list.d/runner_gitlab-runner.list...done.
Importing packagecloud gpg key... Packagecloud gpg key imported to /etc/apt/keyrings/runner_gitlab-runner-archive-keyring.gpg
done.
Running apt-get update... done.

The repository is setup! You can now install packages.
```

Installing GitLab Runner:

The following command was run to confirm the GitLab Runner installation.

```
hasib@runner-24:~$ sudo apt install -y gitlab-runner
```

The output showing *gitlab-runner* is already the newest version (18.8.0-1) means that GitLab Runner was already installed on the system and is currently at the latest/stable version.

```
hasib@runner-24:~$ sudo apt install -y gitlab-runner
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gitlab-runner is already the newest version (18.8.0-1).
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
```

Enabling and starting GitLab Runner service:

The following command is used to enable and start the GitLab Runner service permanently. The enable option ensures that the runner starts automatically even if the VM reboots, and the *--now* option ensures that the service starts immediately.

```
hasib@runner-24:~$ sudo systemctl enable --now gitlab-runner
```

The service status has been verified by running the following command.

```
hasib@runner-24:~$ sudo systemctl status gitlab-runner --no-pager
```

The output *Loaded: loaded ... enabled* shows that the service is properly registered in *systemd* and enabled for boot-time auto-start. The *Active: active (running*) line is the most important piece of evidence, as it confirms that the Runner daemon is currently live. *Running in system-mode* means that the runner was operating as a system-level runner.

```
hasib@runner-24:~$ sudo systemctl status gitlab-runner --no-pager
● gitlab-runner.service - GitLab Runner
     Loaded: loaded (/etc/systemd/system/gitlab-runner.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-02-05 22:22:24 MSK; 1h 32min ago
   Main PID: 8103 (gitlab-runner)
      Tasks: 8 (limit: 4544)
     Memory: 17.7M (peak: 23.8M)
        CPU: 6.412s
     CGroup: /system.slice/gitlab-runner.service
             └─8103 /usr/bin/gitlab-runner run --config /etc/gitlab-runner/config.toml --working-directory /home/gitlab-runner --service gitlab-runner --user …

Feb 05 22:23:55 runner-24 gitlab-runner[8103]: Configuration loaded                    builds=0 max_builds=1
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: WARNING: CONFIGURATION: Long polling issues detected.
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: Issues found:
Feb 05 22:23:55 runner-24 gitlab-runner[8103]:   - Request bottleneck: 1 runners have request_concurrency=1, causing job delays during long polling
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: This can cause job delays matching your GitLab instance's long polling timeout.
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: Recommended solutions:
Feb 05 22:23:55 runner-24 gitlab-runner[8103]:   1. Increase 'request_concurrency' to 2-4 for 1 runners currently using request_concurrency=1
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: Note: The 'FF_USE_ADAPTIVE_REQUEST_CONCURRENCY' feature flag can help automatically adjust request_c…n workload.
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: This message will be printed each time the configuration is reloaded if the issues persist.
Feb 05 22:23:55 runner-24 gitlab-runner[8103]: See documentation: https://docs.gitlab.com/runner/configuration/advanced-configuration.html#long-pol…ax_builds=1
Hint: Some lines were ellipsized, use -l to show in full.
```

Installing Docker:

Docker has been installed and started on VM2 to meet the requirements of CI jobs. Although GitLab uses Runner Shell executor, there will be a Docker image build and push stage in the CI pipeline, so Docker must be present on the Runner VM. For this reason, Docker engine and *Compose v2* have been ensured by running the following command.

```
hasib@runner-24:~$ sudo apt install -y docker.io docker-compose-v2
```

Since the output shows Docker and Docker Compose already installed, there is no need to install anything new. Similarly, the Docker service is enabled and running.

```
hasib@runner-24:~$ sudo apt install -y docker.io docker-compose-v2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (28.2.2-0ubuntu1~24.04.1).
docker-compose-v2 is already the newest version (2.37.1+ds1-0ubuntu2~24.04.1).
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
```

The Docker service is immediately started and set to boot-time auto-start with the following command.

```
hasib@runner-24:~$ sudo systemctl enable --now docker
```

Configuring Docker permissions for runner:

To set Docker permissions correctly, both *gitlab-runner* and user have been added to the docker group. The following command has given Docker access to the runner system user, so that Docker commands can be run during the CI pipeline.

```
hasib@runner-24:~$ sudo usermod -aG docker gitlab-runner
```

The following command is used so that the user can test or debug by running the Docker command manually.

```
hasib@runner-24:~$ sudo usermod -aG docker hasib
```

The following command was run to apply the new group permissions immediately without logout or reboot. This allows Docker commands to be used immediately without *sudo.*

```
hasib@runner-24:~$ newgrp docker
```

Verifying Docker installation:

The following three commands were run to verify that Docker is properly installed and usable.

```
hasib@runner-24:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

If the versions of Docker engine and *Docker Compose* are printed in the first two commands, the installation was successful and the tools are ready to use.

```
hasib@runner-24:~$ docker compose version
Docker Compose version 2.37.1+ds1-0ubuntu2~24.04.1
```

If you see an empty list after running the command below, it means that there are no containers running at the moment. This is completely normal and proves that the Docker daemon is running properly, it's just that no containers have been started yet.

```
hasib@runner-24:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Registering GitLab Runner:

To associate the GitLab Runner with the project, the runner registration was done using the Shell executor. The runner token was collected from the GitLab UI Admin Area CI/CD Runners section, which is required to authenticate the runner with the GitLab server. Then the following command was run to set the GitLab instance URL to *http://24.sne.com/* so that the runner connects to the correct GitLab server.

```
hasib@runner-24:~$ sudo gitlab-runner register --url "http://24.sne.com/" --registration-token "4STDw_TKsm2b2rgQ7Xg6"
```

During registration, the token input is the secret token obtained from GitLab. The runner description is set to *runner-24*, which is the identity of the runner in the GitLab UI. The tags are set *to 24-runner*, so that this runner can be specifically targeted in the CI pipeline. The executor is selected as shell, which means that CI jobs are executed directly in the local bash shell of VM2.

```
hasib@runner-24:~$ sudo gitlab-runner register --url "http://24.sne.com/" --registration-token "4STDw_TKsm2b2rgQ7Xg6"
Runtime platform                                    arch=amd64 os=linux pid=8113 revision=9ffb4aa0 version=18.8.0
Running in system-mode.

Enter the GitLab instance URL (for example, https://gitlab.com/):
[http://24.sne.com/]: http://24.sne.com/
Enter the registration token:
[4STDw_TKsm2b2rgQ7Xg6]: 4STDw_TKsm2b2rgQ7Xg6
Enter a description for the runner:
[runner-24]: runner-24
Enter tags for the runner (comma-separated):
24-runner
Enter optional maintenance note for the runner:

WARNING: Support for registration tokens and runner parameters in the 'register' command has been deprecated in GitLab Runner 15.6 and will be replaced with s
pport for authentication tokens. For more information, see https://docs.gitlab.com/ci/runners/new_creation_workflow/
Registering runner... succeeded                     correlation_id=01KGQM5716QF7NFBE1PEPM4KZW runner=4STDw_TKs
Enter an executor: custom, shell, parallels, docker, docker-windows, kubernetes, docker-autoscaler, ssh, virtualbox, docker+machine, instance:
shell
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
```

Verifying Runner status:

The following two commands were run to verify that the Runner was registered correctly and could communicate with the GitLab server.

```
hasib@runner-24:~$ sudo gitlab-runner verify
```

Running the following command results in the output *Verifying runner... is alive*, indicating that the runner has successfully communicated with GitLab and authentication is OK.

```
hasib@runner-24:~$ sudo gitlab-runner verify
Runtime platform                                    arch=amd64 os=linux pid=8126 revision=9ffb4aa0 version=18.8.0
Running in system-mode.

Verifying runner... is alive                        correlation_id=01KGQM61HYYAS8A81JF7SRP2H9 runner=ypaxcjPzt
```

By running the following command, you can verify that the runner is properly registered, is using shell as the executor, and is associated with the correct GitLab instance URL by seeing the output *runner-24 Executor=shell URL=http://24.sne.com/* . This proves that the Runner is now fully ready to execute CI/CD jobs.

```
hasib@runner-24:~$ sudo gitlab-runner list
```

```
hasib@runner-24:~$ sudo gitlab-runner list
Runtime platform                                arch=amd64 os=linux pid=8133 revision=9ffb4aa0 version=18.8.0
Listing configured runners                      ConfigFile=/etc/gitlab-runner/config.toml
runner-24                                       Executor=shell Token=glrtr-ypaxcjPztu3ZqdW7JpSu URL=http://24.sne.com/
```

Runner configuration file location:

The following command was run to verify that the runner configuration was saved correctly. This file is the main configuration of GitLab Runner, where all the important information of the registered runner is stored.

```
hasib@runner-24:~$ sudo cat /etc/gitlab-runner/config.toml
```

The [[runners]] section of the file shows name = *"runner-24"* and url = *"http://24.sne.com/"* to indicate that the runner's identity and GitLab instance URL are set correctly. The presence of token

= *"glrtr-..."* means that the runner authentication information was successfully saved. And executor = *"shell"* confirms that the runner is using the shell executor.



4. Set up the Deployment Server (VM3).

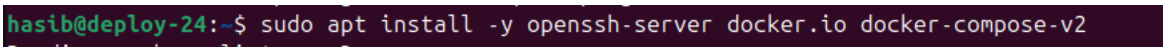
a. Set up authentication of your Gitlab runner to be able to deploy to the deployment server.

Solution:

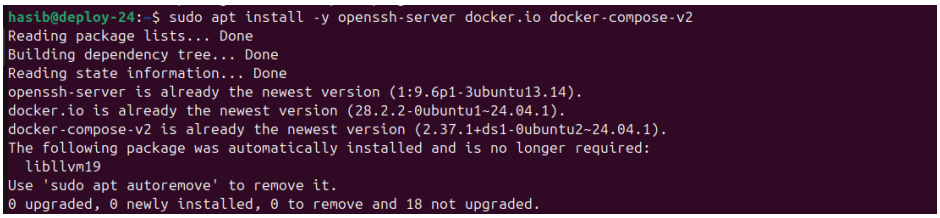
Setting up the Deployment Server:

SSH + Docker Install on VM3:

The following command is run to prepare VM3 as a deployment server. The *openssh-server* is required here so that the SSH daemon is running on VM3 and the runner can easily login remotely during the CI/CD pipeline. The docker.io is installed because the Docker container will run on the deployment server and the web application is deployed there. The *docker-compose-v2* is kept to make it easier to do compose-based deployments in the future.



The output shows *openssh-server* is already the newest version, *docker.io* is already the newest version and *docker-compose-v2* is already the newest version, meaning that all the required packages were already installed and the system is in a deploy-ready state.



Then the following command is used to permanently enable the SSH service. Here the enable option keeps SSH running even after rebooting and the *--now* option starts the SSH daemon immediately.

```
hasib@deploy-24:~$ sudo systemctl enable --now ssh
```

If the output shows Synchronizing state of *ssh.service...,* it means that the SSH service has been successfully enabled.

```
hasib@deploy-24:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
```

The Docker service is enabled and started by running the following command. The reason for this is that the Docker daemon must always be running on the deployment server, because the CI/CD deploy job runs and manages containers directly using Docker commands.

```
hasib@deploy-24:~$ sudo systemctl enable --now docker
```

Deploying user create and docker group:

A dedicated deploy user has been created on VM3 to run deployment tasks separately and securely. The new user is created with the *sudo adduser --gecos* "" deploy command, with the home directory set to /home/deploy and a password configured.

```
hasib@deploy-24:~$ sudo adduser --gecos "" deploy
```

Instead of deploying directly with the root user, a separate user is used, and CI/CD deploy jobs work via SSH with this deploy user.

```
hasib@deploy-24:~$ sudo adduser --gecos "" deploy
info: Adding user `deploy' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `deploy' (1001) ...
info: Adding new user `deploy' (1001) with group `deploy (1001)' ...
info: Creating home directory `/home/deploy' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
info: Adding new user `deploy' to supplemental / extra groups `users' ...
info: Adding user `deploy' to group `users' ...
```

Then the deploy user is added to the docker group using the following command. This is because docker group membership is required to run Docker commands in the deploy stage. With this setup, the deploy user is now ready to deploy applications using Docker.

```
hasib@deploy-24:~$ sudo usermod -aG docker deploy
```

Creating an SSH key with the gitlab-runner user on VM2:

An SSH key has been created with the *gitlab-runner* user on VM2, so that the runner can securely SSH into VM3 without a password during the CI/CD pipeline. Using the following command, the key has been created directly in the name of the *gitlab-runner* user, which is very important for automation.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H ssh-keygen -t ed25519 -N '' -f /home/gitlab-runner/.ssh/id_ed25519
```

Here *-t ed25519* is taken because it is a modern and secure SSH key type. With -N ″the passphrase is left blank, so that automation is possible without any manual input. And with *-f /home/gitlab-runner/.ssh/id_ed25519* the location of the key file is clearly defined, that is, the password or passphrase cannot be kept in CI/CD automation, so this setup is mandatory.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H ssh-keygen -t ed25519 -N '' -f /home/gitlab-runner/.ssh/i
[sudo] password for hasib:
Generating public/private ed25519 key pair.
Created directory '/home/gitlab-runner/.ssh'.
Your identification has been saved in /home/gitlab-runner/.ssh/id_ed25519
Your public key has been saved in /home/gitlab-runner/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:RNFmMogbwN23DT2GOwQPafW+c8TjjfoTDko1lVBzFJg gitlab-runner@runner-24
The key's randomart image is:
+--[ED25519 256]--+
| ..o oo=+=.oo=+. |
|  . + =+B.B Eo   |
|     + oo@.o     |
|     .  .+.+.    |
|        So..+    |
|         . .+.+  |
|        . .oo+.. |
|         .  +o   |
|           ....  |
+----[SHA256]-----+
```

This command creates a private key at */home/gitlab-runner/.ssh/id_ed25519* and a corresponding public key at */home/gitlab-runner/.ssh/id_ed25519.pub*. The output shows the fingerprint and *randomart,* which are the security verification information for the key.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H cat /home/gitlab-runner/.ssh/id_ed25519.pub
```

Then the public key is obtained using the following command. This public key is authorized for the deploy user of VM3 in the next step, so that passwordless SSH from the runner to VM3 is possible.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H cat /home/gitlab-runner/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMYxCwJTJXTsodF3yjxjxIDL9v/d2FJ5wIicWo92N3Wd gitlab-runner@runner-24
```

Inserting authorized_keys in VM3:

Authorized_keys has been configured to enable passwordless SSH for the deploy user in VM3. The .ssh directory has been created with the following command, as this directory is required for SSH authentication to work.

```
hasib@deploy-24:~$ sudo -u deploy -H mkdir -p /home/deploy/.ssh
```

Then, the public key of the *gitlab-runner* user created in VM2 is pasted there using the following command. Having this public key means that *gitlab-runner@runner-24* has the matching private key, so that VM can SSH into VM3 as the deploy user without a user password.

```
hasib@deploy-24:~$ sudo -u deploy -H nano /home/deploy/.ssh/authorized_keys
```

Paste key:

```
  GNU nano 7.2                          /home/deploy/.ssh/authorized_keys *
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMYxCwJTJXTsodF3yjxjxIDL9v/d2FJ5wIicWo92N3Wd gitlab-runner@runner-24
```

Permissions:

Then the correct permission has been set as per the security requirement of SSH. The ownership has been fixed with the following command.

```
hasib@deploy-24:~$ sudo chown -R deploy:deploy /home/deploy/.ssh
```

The following command has only owner access to the .ssh directory.

```
hasib@deploy-24:~$ sudo chmod 700 /home/deploy/.ssh
```

The *authorized_keys* file is read/write only for the owner with the following command.

```
hasib@deploy-24:~$ sudo chmod 600 /home/deploy/.ssh/authorized_keys
```

SSH test (Runner VM3):

SSH connectivity was tested from VM2 to VM3, just like how the runner would work. Using the command below means that the test was run as the *gitlab-runner* user, just like in the CI/CD pipeline. *SSHed* into VM3 as the deploy user with *deploy@deploy-24*, and the *StrictHostKeyChecking=accept-new* option did not block the *known_hosts* prompt automation on the first connection. The SSH command was followed by "*hostname && docker --version*", which checks two things at once, that is, that the login was successful and that the deploy server has permission to run Docker commands.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H ssh -o StrictHostKeyChecking=accept-new deploy@deploy-24 "hostname && docker --version"
```

The output shows Warning: Permanently added *'deploy-24'* to the list of known hosts. This means that the host key has been accepted and added to *known_hosts*. Showing *deploy-24* means that the correct VM was connected, and *Docker version 28.2.2* proves that the Docker command was runnable and the deploy permission was correct.

```
hasib@runner-24:~$ sudo -u gitlab-runner -H ssh -o StrictHostKeyChecking=accept-new deploy@deploy-24 "hostname && docker --version"
Warning: Permanently added 'deploy-24' (ED25519) to the list of known hosts.
deploy-24
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Task 2: Create CI/CD Pipeline

1. Clone the project you have created in step 1.2.g.

Solution:

Cloning repo:

To start working on VM2, first go to the home directory with the following command, so that the cloned repository is clearly in the *~/24*-repo folder.

```
hasib@runner-24:~$ cd ~
```

Then the 24-repo project was cloned from the GitLab server (VM1) by running the following command. Since HTTP is used here, GitLab naturally asked for a username and password.

```
hasib@runner-24:~$ git clone http://24.sne.com/root/24-repo.git
```

The output shows Cloning into *'24-repo'*... followed by the Username for *'http://24.sne.com'*: root and Password for *'http://root@24.sne.com'*: prompt, which proves that GitLab HTTP authentication worked. After logging in with the root user, remote: Enumerating objects... and Receiving *objects: 100% (3/3*), done. This means that the initial repository objects have been successfully downloaded from GitLab and the clone is complete.

```
hasib@runner-24:~$ git clone http://24.sne.com/root/24-repo.git
Cloning into '24-repo'...
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Then, we entered the repository directory using cd *24-repo*. From now on, all the code, configuration, and CI/CD files will be inside this folder and Git commits and pushes will be done from here.

```
hasib@runner-24:~$ cd 24-repo
hasib@runner-24:~/24-repo$
```

2. Write a simple web application in any programming language. (E.g. Random text or Addition of two numbers)

Solution:

Creating a folder structure:

To organize and future-proof the project, four directories have been created: backend, frontend, tests, and ansible. The backend/ folder contains the Python code for the Flask server, frontend/ contains HTML/JS-based static files, tests/ is used to write automated tests using *pytest*, and the ansible/ folder is used to facilitate adding the Ansible deploy playbook to Task 3. This keeps the project structure clear and makes it easy to extend the CI/CD pipeline.

```
hasib@runner-24:~/24-repo$ mkdir -p backend frontend tests ansible
```

Backend (Flask) - backend/main.py:

This file creates a simple Flask web application.

```
hasib@runner-24:~/24-repo$ nano backend/main.py
```

Initially, os is used to read the environment variable (*APP_VERSION*) and the necessary Flask modules (Flask, request, jsonify, send_from_directory) are imported, so that API responses and static files can be served. The frontend folder is set as a static folder and static content will be served from the root path (/). Version is configured with *VERSION = os.getenv("APP_VERSION", "dev"*"), so that the version can be injected from the container or CI pipeline. In the routing section, the frontend's index.html is served from the / route, the /health endpoint is used to check if the app is alive (CI/CD and monitoring-friendly), the /version endpoint is used for build/version debug, and the /api/add endpoint takes a and b as query parameters and returns their sum in JSON format, along with version information. *app.run(host="0.0.0.0", port=5000)* is used, so that the app accepts requests from all network interfaces. This allows access from outside the container or VM and is ready to expose the service on port 5000.

```
 GNU nano 7.2                                                backend/main.py *
import os
from flask import Flask, request, jsonify, send_from_directory

app = Flask(__name__, static_folder="../frontend", static_url_path="/")
VERSION = os.getenv("APP_VERSION", "dev")

@app.get("/")
def home():
    return send_from_directory(app.static_folder, "index.html")

@app.get("/health")
def health():
    return jsonify(status="ok", version=VERSION)

@app.get("/version")
def version():
    return jsonify(version=VERSION)

@app.get("/api/add")
def add():
    a = int(request.args.get("a", "0"))
    b = int(request.args.get("b", "0"))
    return jsonify(a=a, b=b, sum=a+b, version=VERSION)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)


^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location      M-U Un
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Re
```

Backend requirements - backend/requirements.txt:

The backend/requirements.txt file has been created to manage the Python dependency of the backend and specifies *flask==3.0.3*.

```
hasib@runner-24:~/24-repo$ nano backend/requirements.txt
```

The exact version of Flask has been pinned here, so that the same version is used everywhere - on the local machine, in the CI pipeline, and in production.

```
 GNU nano 7.2                                          backend/requirements.txt *
flask==3.0.3
```

Frontend - frontend/index.html:

The file below creates a simple static frontend that interacts with the backend Flask API. A basic structure is placed inside the HTML, where 24 WebApps (Frontend + Backend) are displayed.

```
hasib@runner-24:~/24-repo$ nano frontend/index.html
```

The first part contains the Check Health button, which calls the /health endpoint of the backend and displays the response in the *<pre id="healthOut">* block. The *<pre>* tag allows the JSON output to be displayed in a readable format. The second part contains two input fields (a and b) and an Add button. Here, when the user inputs a value, the frontend calls the */api/add* endpoint and the resulting JSON result is displayed in the *<pre id="addOut">* block. The app.js script is included, which provides interactivity to the frontend.

```
 GNU nano 7.2                                                frontend/index.html *
<!doctype html>
<html>
<head>
  <meta charset="utf-8" />
  <title>24 WebApp</title>
</head>
<body>
  <h1>24 WebApp (Frontend + Backend)</h1>

  <div>
    <button id="healthBtn">Check Health</button>
    <pre id="healthOut"></pre>
  </div>

  <hr />

  <div>
    <input id="a" type="number" value="2" />
    <input id="b" type="number" value="3" />
    <button id="addBtn">Add</button>
    <pre id="addOut"></pre>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

Frontend JS - frontend/app.js:

The following JavaScript file connects the frontend to the backend Flask API and adds interactivity to the page.

```
hasib@runner-24:~/24-repo$ nano frontend/app.js
```

Initially, the Health and Add buttons and the <*pre*> elements to display the output are retrieved using *getElementById.* In the *healthBtn.onclick* event, the health endpoint of the backend is called using fetch*("/health").* The JSON data received from the response is nicely formatted using *JSON.stringify*(*await r.json(), null, 2*) and displayed in the *healthOut* block. In the *addBtn.onclick* event, the values of a and b are retrieved from the input field and the backend's add API is called using *fetch(\/api/add?a=${a}&b=${b}`).* Here too, the returned JSON data is printed in the addOut` block in a readable format.

```
 GNU nano 7.2                                                frontend/app.js *
const healthBtn = document.getElementById("healthBtn");
const addBtn = document.getElementById("addBtn");
const healthOut = document.getElementById("healthOut");
const addOut = document.getElementById("addOut");

healthBtn.onclick = async () => {
  const r = await fetch("/health");
  healthOut.textContent = JSON.stringify(await r.json(), null, 2);
};

addBtn.onclick = async () => {
  const a = document.getElementById("a").value;
  const b = document.getElementById("b").value;
  const r = await fetch(`/api/add?a=${a}&b=${b}`);
  addOut.textContent = JSON.stringify(await r.json(), null, 2);
};

^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Lin
```

3. Create CI/CD pipeline (.gitlab-ci.yml)

a. CI stages of the pipeline should:

i. Build the application

ii. Run test (to check the application works ok)

iii. Build docker image (Note: you need Dockerfile)

iv. Push to your docker hub account.

b. CD stages of the pipeline should: i. Pull the docker image and deploy it on the deployment server

Solution:

Test case - tests/test_api.py:

The file below contains a simple automated test for the backend Flask API. Here, the Flask application is imported using from *backend.main* import app and a test client is created using Flask's built-in *app.test_client()*.

```
hasib@runner-24:~/24-repo$ nano tests/test_api.py
```

This client can simulate API calls, without the need to run a separate server. This is useful for CI/CD pipelines. The *test_health()* function calls the */health* endpoint and verifies that the response has an HTTP status code of *200 (OK)* and the status value in the JSON output is "ok". The *test_add()* function calls the */api/add?a=2&b=3* endpoint and verifies that the status code is 200 and the sum value in the JSON response is correctly 5.

```
  GNU nano 7.2                                          tests/test_api.py *
from backend.main import app

def test_health():
    c = app.test_client()
    r = c.get("/health")
    assert r.status_code == 200
    assert r.json["status"] == "ok"

def test_add():
    c = app.test_client()
    r = c.get("/api/add?a=2&b=3")
    assert r.status_code == 200
    assert r.json["sum"] == 5
```

Dockerfile:

The *Dockerfile* below is designed to containerize backend, frontend, and tests together, so that a reproducible Docker image can be built in a CI/CD pipeline.

```
hasib@runner-24:~/24-repo$ nano Dockerfile
```

FROM python:3.12-slim is used to get a lightweight Python base image, which is fast to pull and has a small final image size. By setting *WORKDIR /app*, all work inside the container is done inside the */app* directory. *COPY backend/requirements.txt* . This was done earlier so that Docker layer caching worked properly. Then the necessary Python dependencies were installed with *RUN pip install --no-cache-dir -r requirements.txt* , where using *--no-cache-dir* does not keep the pip cache and the image is small. Then the entire application code was copied into the container with the lines *COPY backend ./backend*, *COPY frontend ./frontend* and COPY tests ./tests . A default version was set *with ENV APP_VERSION=dev* . *EXPOSE 5000* is given as metadata for the port on which the application will run. CMD *["python", "backend/main.py"]* specifies that the Flask application will be launched when the container starts.

```
 GNU nano 7.2                                                Dockerfile *
FROM python:3.12-slim

WORKDIR /app
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY backend ./backend
COPY frontend ./frontend
COPY tests ./tests

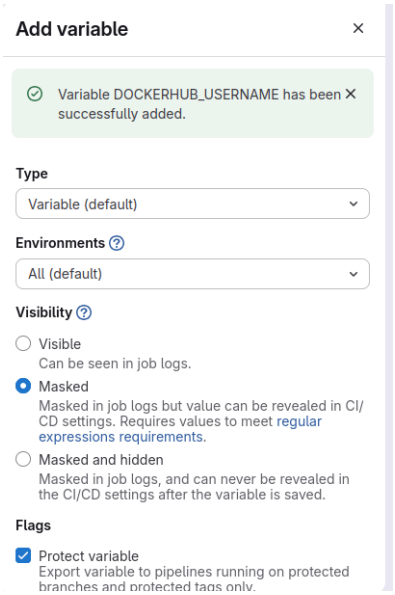
ENV APP_VERSION=dev
EXPOSE 5000
CMD ["python", "backend/main.py"]
```

Setting GitLab CI Variables:

GitLab UI:

This step sets GitLab CI/CD Variables, because the Docker image build, push, and deploy stages of the pipeline require a login to Docker Hub. As per security best practices, Docker Hub username, password, or token should never be hardcoded in the repo or *.gitlab-ci.yml*. Therefore, GitLab's CI/CD Variables are used, which are safely injected as environment variables in the pipeline runtime.

In GitLab UI, open the 24-repo project and go to the Variables section from Settings to CI/CD. The variables are added. The *DOCKERHUB_USERNAME* variable holds the Docker Hub username and is used in docker login *-u* . The most important variable is *DOCKERHUB_TOKEN*, where the Docker Hub access token or password is stored and is masked so that the secret is never revealed in the pipeline log. This token is used in the pipeline via docker login *--password-stdin* . The third variable *DOCKER_IMAGE* is set (e.g. *hasib/24-webapp*), so that a consistent image name can be used across Docker image builds, tags, pushes, and pulls.



Ansible files:

inventory.ini

This inventory file is created to define target hosts for the Ansible deploy playbook. Here *[deploy]* is an inventory group name, which can be easily targeted by typing hosts: deploy in the playbook.

```
hasib@runner-24:~/24-repo$ nano ansible/inventory.ini
```

deploy-24 is given as the hostname of VM3, which is already mapped in the */etc/hosts* file, so Ansible can resolve it. *ansible_user=deploy* specifies that the deploy user is used when *SSHing* into VM3. This causes Ansible deploy tasks to be performed by the dedicated deploy user, not root.

```
  GNU nano 7.2                                    ansible/inventory.ini
[deploy]
deploy-24 ansible_user=deploy
```

deploy.yml

This Ansible playbook is used to deploy 24 webapps to VM3. The inventory *[deploy]* group is targeted with hosts: deploy and become: true because Docker and port binding may require root privileges.

```
hasib@runner-24:~/24-repo$ nano ansible/deploy.yml
```

Logged in securely (*--password-stdin*) with Docker Hub username and token from GitLab CI/CD Variables. VM3 then pulls the latest image from Docker Hub, so that the newest build is always deployed. If there is an old *24-webapp* container, it is removed to avoid conflicts, and

ignore_errors: true is used to prevent the playbook from failing even if the container is not there. Then a new container is run, where the app is exposed with *-p 8080:5000* and *CI_COMMIT_SHORT_SHA* is sent as *APP_VERSION,* so that any commit deployed can be traced. Health is checked with curl /health, where if the response is not correct, the playbook and pipeline fail, which is correct CI/CD behavior.



.gitlab-ci.yml (CI + CD together):

CI (build+test) + CD (docker push + ansible deploy) all done with the *same .gitlab-ci.yml*.



CI to CD flow is organized with stages. If *IMAGE_TAG=$CI_COMMIT_SHORT_SHA* is given, each commit image is different (traceability) + latest is kept as "current build". Dependency install is faster because of pip cache in cache. Build job creates *.venv* artifact, test job reuses that and Docker jobs run only on master/develop, deploy only on master. Docker Hub creds */DOCKER_IMAGE* is not hardcoded, it comes at runtime from GitLab UI Variables. Ansible is installed in *deploy_ansible* job so that pipeline can run even if it is not installed on VM2 before. Docker image is built on master/develop branch.

```
 GNU nano 7.2                                                    .gitlab-ci.yml
stages:
  - build
  - test
  - docker_build
  - docker_push
  - deploy

variables:
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
  # Non-secret → file এ রাখা যায়
  DOCKERHUB_USERNAME: "yourdockername"
  DOCKER_IMAGE: "yourdockername/24-webapp"

cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - .cache/pip

build:
  stage: build
  script:
    - python3 --version
    - python3 -m venv .venv
    - . .venv/bin/activate
    - pip install --upgrade pip
    - pip install -r backend/requirements.txt
  tags:
    - "24-runner"
```

```
  tags:
    - "24-runner"
  artifacts:
    when: always
    expire_in: 1 week
    paths:
      - .venv/
      - backend/requirements.txt

test:
  stage: test
  needs: ["build"]
  dependencies: ["build"]
  script:
    - . .venv/bin/activate
    - export PYTHONPATH="$CI_PROJECT_DIR"
    - pip install pytest==8.2.2
    - pytest -q --junitxml=report.xml
  tags:
    - "24-runner"
  artifacts:
    when: always
    reports:
      junit: report.xml
    paths:
      - report.xml


^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line
```

```
docker_build:
  stage: docker_build
  needs: ["test"]
  script:
    - 'test -n "$DOCKER_IMAGE" || (echo "ERROR: DOCKER_IMAGE is empty"; exit 1)'
    - 'echo "Using image: $DOCKER_IMAGE:$IMAGE_TAG"'
    - docker version
    - docker build -t "$DOCKER_IMAGE:$IMAGE_TAG" .
    - docker tag "$DOCKER_IMAGE:$IMAGE_TAG" "$DOCKER_IMAGE:latest"
  tags:
    - "24-runner"
  rules:
    - if: '$CI_COMMIT_BRANCH == "master" || $CI_COMMIT_BRANCH == "develop"'

docker_push:
  stage: docker_push
  needs: ["docker_build"]
  script:
    - 'test -n "$DOCKERHUB_TOKEN" || (echo "ERROR: DOCKERHUB_TOKEN is empty (set it in GitLab CI/CD variables)"; exit 1)'
    - echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin
    - docker push "$DOCKER_IMAGE:$IMAGE_TAG"
    - docker push "$DOCKER_IMAGE:latest"
  tags:
    - "24-runner"
  rules:
    - if: '$CI_COMMIT_BRANCH == "master" || $CI_COMMIT_BRANCH == "develop"'
```

```
deploy_ansible:
  stage: deploy
  needs: ["docker_push"]
  script:
    - ansible --version
    - ansible -i ansible/inventory.ini deploy -m ping
    - ansible-playbook -i ansible/inventory.ini ansible/deploy.yml
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'



^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify
```

Commit and Push:

I have run two commands to add all the code and configuration to the Git repository.

```
hasib@runner-24:~/24-repo$ git add .
```

```
hasib@runner-24:~/24-repo$ git commit -m "Frontend+Backend app + unified CI/CD + Ansible deploy"
```

The output showing 9 files changed, 189 insertions(+) means that the new backend/frontend code, tests, *Dockerfile*, *.gitlab-ci.yml,* and Ansible inventory and playbook-all have been successfully committed.

```
hasib@runner-24:~/24-repo$ git commit -m "Frontend+Backend app + unified CI/CD + Ansible deploy"
[main c9408f5] Frontend+Backend app + unified CI/CD + Ansible deploy
 9 files changed, 189 insertions(+)
 create mode 100644 .gitlab-ci.yml
 create mode 100644 Dockerfile
 create mode 100644 ansible/deploy.yml
 create mode 100644 ansible/inventory.ini
 create mode 100644 backend/main.py
 create mode 100644 backend/requirements.txt
 create mode 100644 frontend/app.js
 create mode 100644 frontend/index.html
 create mode 100644 tests/test_api.py
```

Then the branch was renamed, because the master branch had to be used according to the lab requirement.

```
hasib@runner-24:~/24-repo$ git branch
* main
```

The local branch has been changed from main to master with the following command.

```
hasib@runner-24:~/24-repo$ git branch -m main
```

Then, the master branch was pushed to the remote GitLab repository by running the following command.

```
hasib@runner-24:~/24-repo$ git push -u origin master
```

Showing *master -> master* in the output means that the branch has been created on the remote and being set up to track 'origin/master' will make pushing/pulling easier in the future.

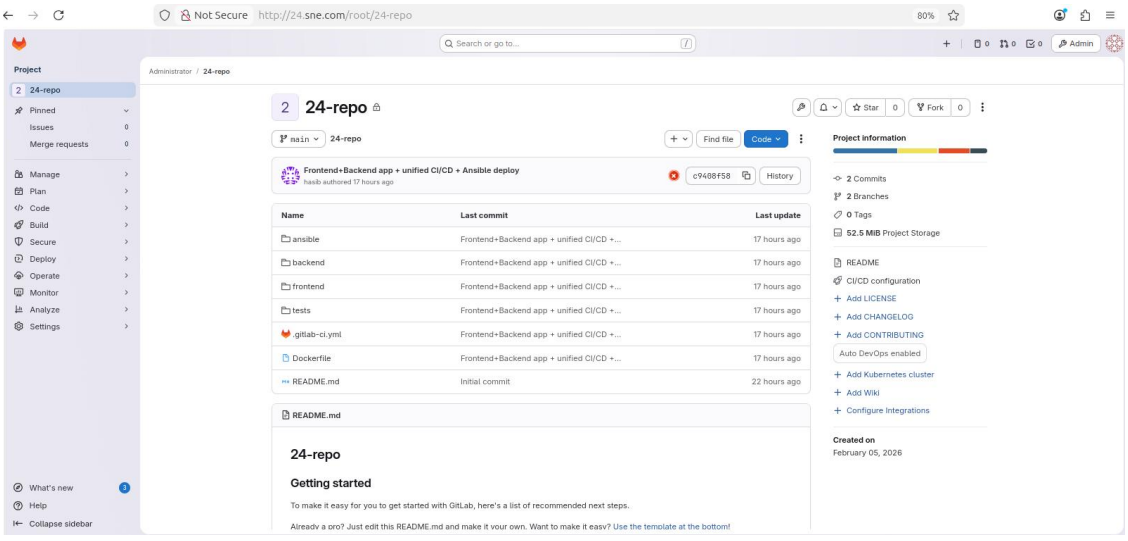
```
hasib@runner-24:~/24-repo$ git push -u origin master
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for master, visit:
remote:   http://24.sne.com/root/24-repo/-/merge_requests/new?merge_request%5Bsource_branch%5D=master
remote:
To http://24.sne.com/root/24-repo.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

View Pipeline:

To verify the pipeline, open the latest pipeline by going to Project, Build, Pipelines in the GitLab UI.

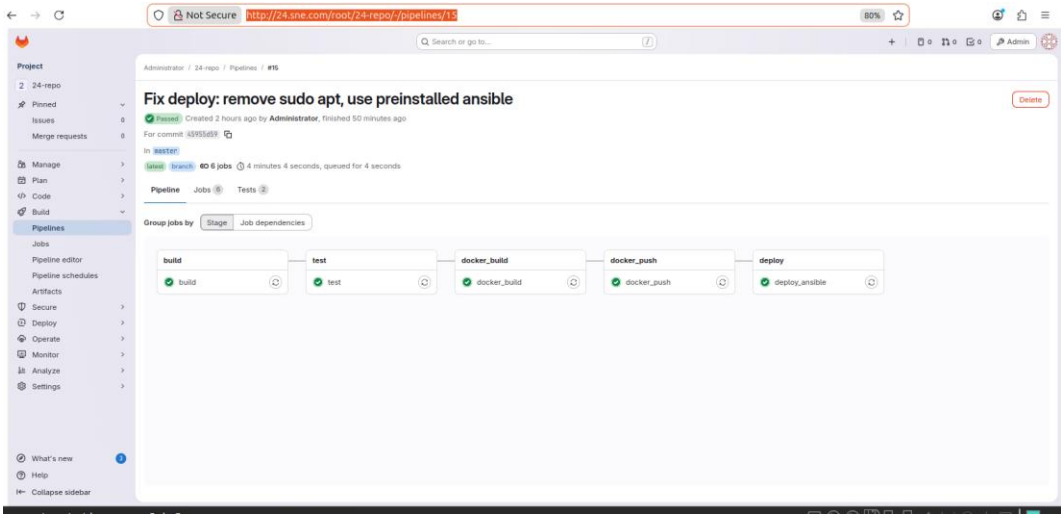
GitLab UI:

Project 24-repo:



Pipelines:

All stages being green means that build, test, Docker image build/push, and Ansible deploy all steps have been completed successfully.



4. Validate that the deployment is successful by accessing the web app via the browser on deployment server side.

Solution:

Deployment verify:

The following command was run to verify that the API is working directly from VM3. Here localhost is used because the container is bound to port *8080* of VM3 in the Ansible deploy playbook.

```
deploy@deploy-24:~$ curl http://localhost:8080/health
```

The output shows *{"status":"ok","version":"45955d59"}* which means the app is running successfully and the request has been served correctly. The status*: "ok"* proves that the backend service is in a healthy state. And the version: "*45955d59*" shows that *APP_VERSION=$CI_COMMIT_SHORT_SHA* was set when running the container in the deploy stage, so the SHA of the deployed commit is shown here directly.

```
deploy@deploy-24:~$ curl http://localhost:8080/health
{"status":"ok","version":"45955d59"}
```

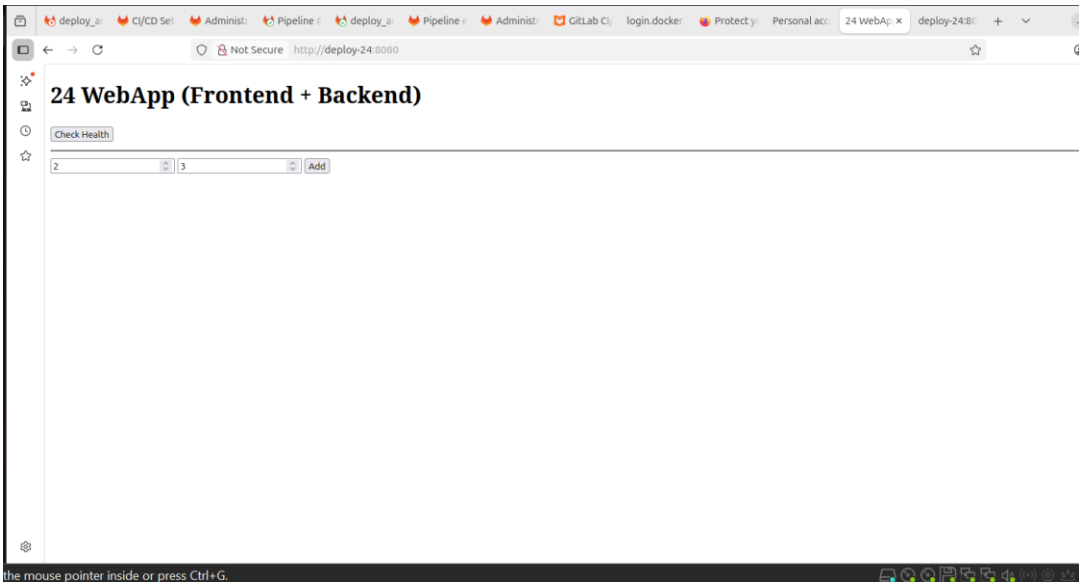
The following command was run to verify the addition API from VM3.

```
deploy@deploy-24:~$ curl "http://localhost:8080/api/add?a=10&b=20"
```

The output of *{"a":10,"b":20,"sum":30,"version":"45955d59"}* means that the app correctly received a=10 and b=20 as query parameters and returned the sum 30 as per the business logic, which proves that the application logic is working properly. version: 45955d59 again confirms that the same deployed build (commit SHA) is running on VM3 and that there is consistent deployment across all endpoints.

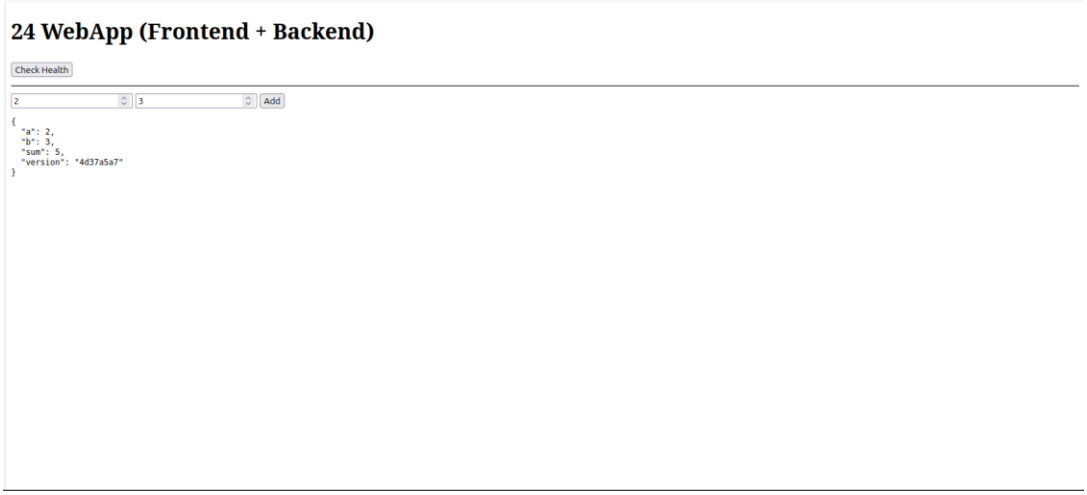
```
deploy@deploy-24:~$ curl "http://localhost:8080/api/add?a=10&b=20"
{"a":10,"b":20,"sum":30,"version":"45955d59"}
deploy@deploy-24:~$
```

Opening *http://deploy-24:8080/* from the browser loads a page titled "24 WebApp (Frontend + Backend)" and shows the Health check button and input fields. This proves that the Docker container's port mapping *(8080:5000)* is working properly, the frontend's static files are being served properly, and the web application is accessible from the deployment server (VM3) externally.

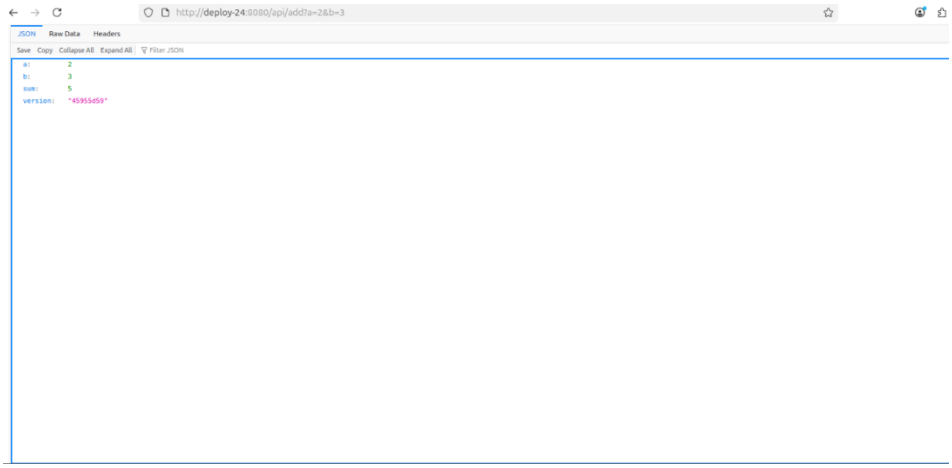


View output by clicking the button:

Clicking the Check Health button in the browser resulted in a call to the /*health* endpoint of the backend from the frontend and the JSON response was displayed directly on the page. This proves that the integration between the frontend and backend worked properly and the entire flow is functional end-to-end.



Opening *http://deploy-24:8080/api/add?a=2&b=3* in a browser results in a JSON response of sum = 5. This means that the backend API call from the browser was successful, the application logic worked correctly, and the deployed web app is fully usable.



Task 3: Polish the CICD

5. Update the CD stages to be able to deploy the web application using Ansible.

Solution:

Creating Ansible files in Repo on VM2 (runner-24): Creating the following folder in the ansible/ directory keeps all Ansible-related files (inventory, playbook) in one place and keeps the Repository clean and maintainable.



Creating inventory.ini:

This inventory file is created to define the deployment target for the Ansible playbook.

```
hasib@runner-24:~/24-repo$ nano ansible/inventory.ini
```

[deploy] is the inventory group name, which is easily targeted in the playbook by typing *hosts: deploy* . deploy-24 is a logical host name, used here for readability and ease of management. *ansible_host=10.10.10.13* specifies the actual IP address of VM3, so that Ansible can be sure to connect to the correct machine even if there are problems with DNS or /etc/hosts. *ansible_user*=deploy specifies that the deploy user is used when *SSHing* into VM3.

```
  GNU nano 7.2                                          ansible/inventory.ini
[deploy]
deploy-24 ansible_host=10.10.10.13 ansible_user=deploy
```

Creating the deploy.yml playbook:

The following playbook is written to deploy 24 webapps to VM3. The name line makes the purpose of the playbook clear. The *[deploy]* group (VM3) of the inventory is being targeted with *hosts: deploy* . Become: true is kept because root privilege is required to run the Docker command and bind the port.

```
hasib@runner-24:~/24-repo$ nano ansible/deploy.yml
```

The Docker Hub username and token are retrieved from GitLab CI/CD Variables using *lookup('env', ...).* Using *--password-stdin* does not leak the token log and VM3 gets permission to pull images from Docker Hub. The latest tag is pulled from Docker Hub, so the latest image built in the pipeline is always deployed. If there is a previous *24-webapp* container, it is removed to avoid name/port conflicts. By setting *ignore_errors: true* , the playbook does not fail even if the container does not exist. The new container is launched in detached mode, with the specified name (*24-webapp*). CI_COMMIT_SHORT_SHA is sent as *APP_VERSION*, so that the deployed build can be traced. The app is exposed on port *8080* of VM3 with *-p 8080:5000*. The /health endpoint is verified with *curl -fsS*. If *HTTP 200* is not returned, the playbook and pipeline fail.



SSH login to VM3:

From VM2, login to VM3 as deploy user by running the following command. Here *deploy@10.10.10.13* means SSH is done as deploy user using IP of VM3.

```
deploy@deploy-24:~$ ssh deploy@10.10.10.13
```

The output shows Welcome to *Ubuntu 24.04.3 LTS* ... which indicates that the SSH login was successful. Similarly, the line *Last login* ... means that the session was established correctly and was now ready to work on VM3.

```
deploy@deploy-24:~$ ssh deploy@10.10.10.13
deploy@10.10.10.13's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-14-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Feb  7 00:21:36 2026 from 10.10.10.13
```

Adding the deploy user to the docker group:

The deploy user is added to the docker group by running the following command on VM3. The *-aG* option is used here so that the docker group is appended to the existing group list and any previous group membership of the deploy user is not deleted.

```
deploy@deploy-24:~$ sudo usermod -aG docker deploy
```

Adding Passwordless sudo rule (*NOPASSWD*):

Passwordless sudo is set for the deploy user by running the following command on VM3. It is best practice to have a separate file in */etc/sudoers.d/,* as it does not require editing the main */etc/sudoers* file and keeps the configuration clean.

```
deploy@deploy-24:~$ echo 'deploy ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/deploy
```

This rule means that the deploy user can run any command with *sudo*, without a password. This is important for CI/CD automation, because the pipeline gets stuck when the sudo password prompt occurs during Ansible deploy. The output shows deploy *ALL=(ALL) NOPASSWD: ALL* , which means the rule file was written successfully and passwordless *sudo* is configured correctly.

```
deploy@deploy-24:~$ echo 'deploy ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/deploy
deploy ALL=(ALL) NOPASSWD: ALL
```

Fixing *sudoers* file permission:

The correct permissions of the *sudoers* rule file are set with the following command. *sudo* checks the permissions of the *sudoers* file very strictly, if the permissions are wrong, it ignores the file for security reasons and the rule may not work. 440 permission means that the owner and group can only read, no write or execute permissions. This is a safe and recommended setting for the sudoers file, so the passwordless *sudo* configuration works properly.

```
deploy@deploy-24:~$ sudo chmod 440 /etc/sudoers.d/deploy
```

Logout/login to apply group changes:

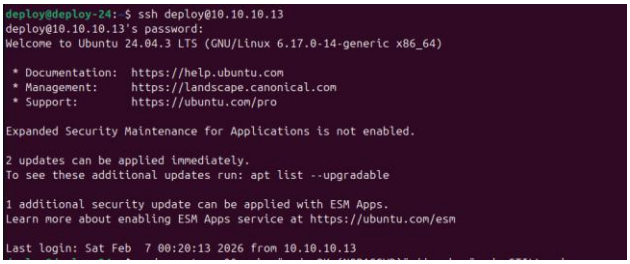
The current SSH session has been exited with the exit command.

```
deploy@deploy-24:~$ exit
logout
Connection to 10.10.10.13 closed.
```

I have login again with the following command.

```
deploy@deploy-24:~$ ssh deploy@10.10.10.13
```

This is because the group membership changed with *usermod -aG* docker deploy only takes effect when a new session is started.



Quick sudo verification:

The following command was run on VM3. Here *sudo -n* means non-interactive mode, if a password is required, *sudo* fails immediately and does not show any prompt. true is a harmless test command, used for verification only.

```
deploy@deploy-24:~$ sudo -n true && echo "sudo OK (NOPASSWD)" || echo "sudo STILL asks password"
```

The output below shows that the deploy user is now able to run *sudo* without a password, even in a non-interactive context. This proves that the "*Missing sudo password"* issue is no longer encountered during the CI/CD pipeline and the passwordless *sudo* configuration has been completed successfully.

```
deploy@deploy-24:~$ sudo -n true && echo "sudo OK (NOPASSWD)" || echo "sudo STILL asks password"
sudo OK (NOPASSWD)
```

Deployment evidence:

Running the following command on VM3 shows that the container named 24-webapp is running using the *luffysungod/24-webapp:latest* image.

```
deploy@deploy-24:~$ docker ps
```

The output shows *0.0.0.0:8080->5000/tcp* which means that port 8080 of VM3 is correctly mapped to port 5000 of the container. This proves that the Docker image was successfully pulled, the container is running, the port mapping is correct, and the specified container name (24-webapp) is active as per the Ansible deploy playbook. This means the deployment is successful.

```
deploy@deploy-24:~$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED       STATUS       PORTS
                                  NAMES
eab73226c909   luffysungod/24-webapp:latest   "python backend/main…"   2 hours ago   Up 2 hours   0.0.0.0:808
0->5000/tcp, [::]:8080->5000/tcp   24-webapp
```

Ansible connectivity proof:

Since Ansible is being used in the Deploy stage, it has been verified that Ansible is installed on runner VM2.

```
hasib@runner-24:~/24-repo$ ansible --version
```

Running the above command shows ansible *[core 2.16.3]* in the output, which proves that Ansible is installed correctly. The same output shows *python version 3.12.3*, which shows that the Python runtime required to run Ansible is available. Also, the path of the Ansible executable is */usr/bin/ansible*, which means that it was reliably usable in the system-level install and CI/CD pipeline.

```
hasib@runner-24:~/24-repo$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/hasib/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/hasib/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

6. Update the pipeline to support multi-branch (e.g. master and develop) and jobs should be triggered based on the specific target branch.

Solution:

Creating develop branch:

Running the following command created a new develop branch and immediately switched to that branch.

```
hasib@runner-24:~/24-repo$ git checkout -b develop
```

The new commit is now in the develop branch. This is necessary to follow the multi-branch workflow.

```
hasib@runner-24:~/24-repo$ git checkout -b develop
Switched to a new branch 'develop'
```

Developing branch push to GitLab:

Running the following command creates the develop branch in the GitLab remote repository.

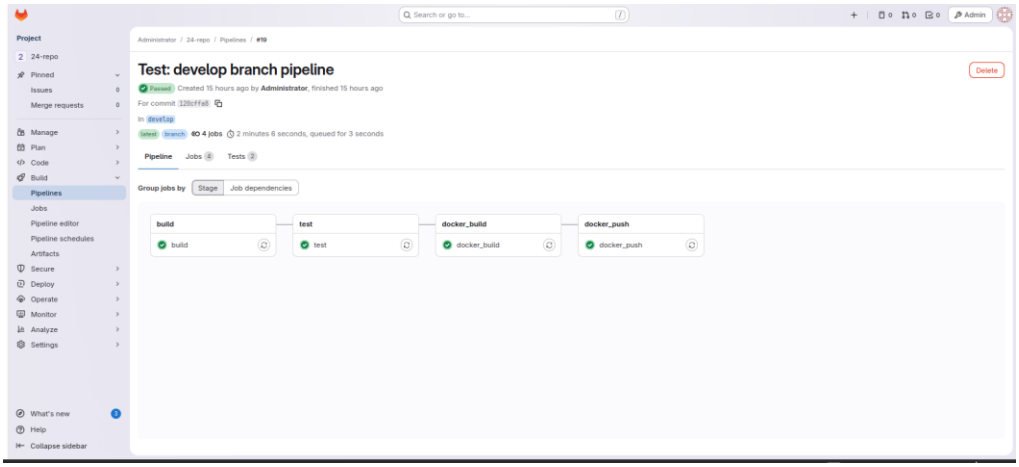
```
hasib@runner-24:~/24-repo$ git push -u origin develop
```

The output shows *[new branch] develop -> develop* which means the new branch was successfully created on the remote. With the *-u* option, the local develop branch now tracks origin/develop.

```
hasib@runner-24:~/24-repo$ git push -u origin develop
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for develop, visit:
remote:   http://24.sne.com/root/24-repo/-/merge_requests/new?merge_request%5Bsource_branch%5D=develop
remote:
To http://24.sne.com/root/24-repo.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

Pipelines:

I have build and all four jobs ran.



Develop branch verification:

Running the following command confirms that it is currently in the develop branch.

```
hasib@runner-24:~/24-repo$ git checkout develop
```

Seeing Switched to branch 'develop' in the output means the branch switch was successful. *M ansible/deploy.yml* and M ansible/inventory.ini were visible, where M means that those files had local changes (modified). This is normal and means that the files have been changed but not yet committed. Also, the develop branch was now ready to sync/track with the remote origin/develop.

```
hasib@runner-24:~/24-repo$ git checkout develop
M       ansible/deploy.yml
M       ansible/inventory.ini
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
```

Develop pipeline trigger with small change:

The following three important commands are executed.

```
hasib@runner-24:~/24-repo$ git add .
```

```
hasib@runner-24:~/24-repo$ git commit -m "Test: develop branch pipeline"
```

The output shows commit ID *([develop 120cffa] ...)* and *push successful*, meaning the change was successfully pushed to GitLab. GitLab automatically triggered the pipeline with this push, which is proof that the develop branch CI worked properly.

Pipeline behavior:

- *build* stage ran
- *test* stage ran
- *docker_build* stage ran
- *docker_push* stage ran
- *deploy_ansible* did not run

According to the rules of the *deploy_ansible* job, deploy is only allowed from the master branch. So, deploy is blocked on the develop branch.

```
hasib@runner-24:~/24-repo$ git push
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 529 bytes | 27.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for develop, visit:
remote:   http://24.sne.com/root/24-repo/-/merge_requests/new?merge_request%5Bsource_branch%5D=develop
remote:
To http://24.sne.com/root/24-repo.git
   4d37a5a..120cffa  develop -> develop
```

7. Update keywords such as cache, artifact, needs, and dependencies to have more control of pipeline execution.

Solution:

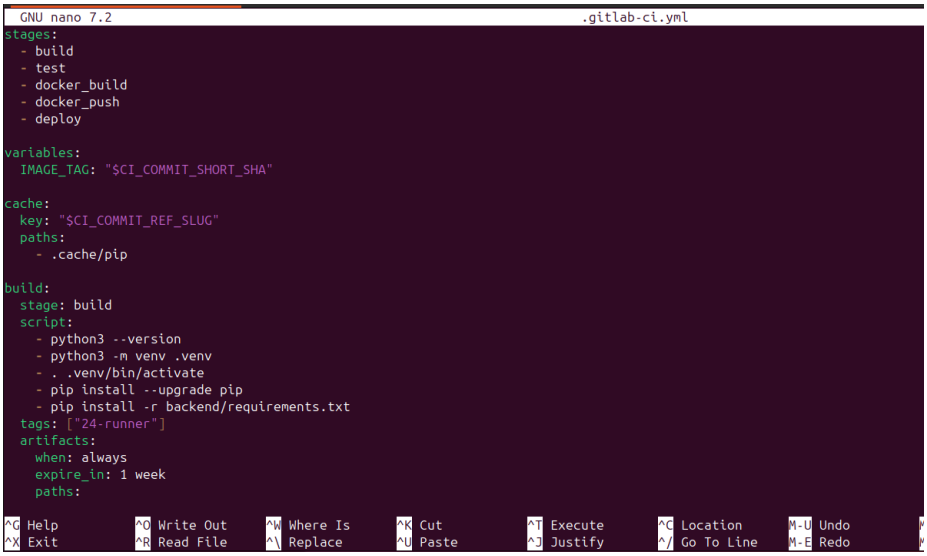
.gitlab-ci.yml:

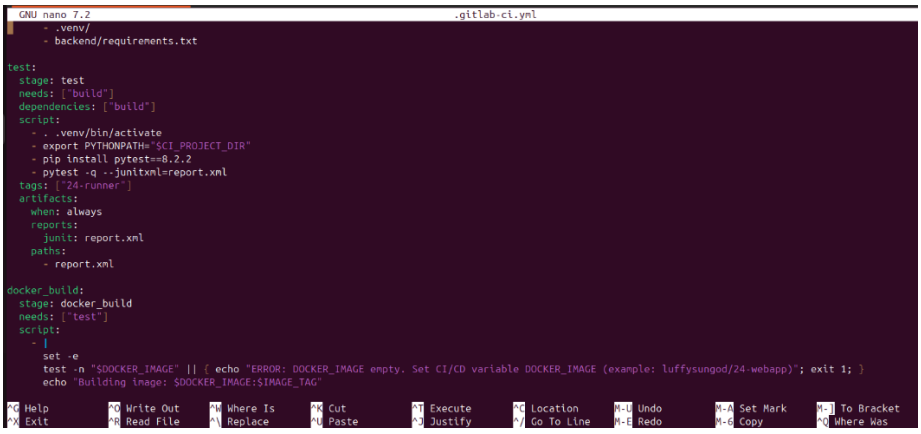
It is placed in the repo root of *VM2 (runner-24): ~/24-repo/.gitlab-ci.yml*

This pipeline is divided into 5 stages. Since *IMAGE_TAG=$CI_COMMIT_SHORT_SHA*, each commit image gets a different tag (traceability), and the latest is the "*current build*".

```
hasib@runner-24:~/24-repo$ nano .gitlab-ci.yml
```

cache: paths: .cache/pip means that the packages downloaded by pip are cached and reused in the next pipeline run. By giving *key: $CI_COMMIT_REF_SLUG*, the cache is branch-wise separate, so that the build is stable and the job completes quickly. *.venv/* artifact is kept in the build job, so that the next job (test) can reuse the same *virtualenv*. By giving *reports: junit: report.xml* in the test job, the results are shown in the Test Report tab in the GitLab UI and it is clear which test has failed. By giving paths: report.xml, the report file is saved. *needs: ["build"] / needs: ["test"]* tells GitLab to start this job only after that dependency job is complete. This reduces stage-based unnecessary waiting, and the pipeline overall is faster. With dependencies: *["build"],* the test job only downloads the artifacts *(here .venv)* of the build job. No unnecessary artifacts need to be downloaded, resulting in less bandwidth, faster execution. *docker_build* and *docker_push* only run on master/develop. *deploy_ansible* only runs on master.





```
GNU nano 7.2                                                    .gitlab-ci.yml
    docker version
    docker build -t "$DOCKER_IMAGE:$IMAGE_TAG" .
    docker tag "$DOCKER_IMAGE:$IMAGE_TAG" "$DOCKER_IMAGE:latest"
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master" || $CI_COMMIT_BRANCH == "develop"'

docker_push:
  stage: docker_push
  needs: ["docker_build"]
  script:
    - |
      set -e
      test -n "$DOCKERHUB_USERNAME" || { echo "ERROR: DOCKERHUB_USERNAME empty"; exit 1; }
      test -n "$DOCKERHUB_TOKEN" || { echo "ERROR: DOCKERHUB_TOKEN empty"; exit 1; }
      echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin
      docker push "$DOCKER_IMAGE:$IMAGE_TAG"
      docker push "$DOCKER_IMAGE:latest"
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master" || $CI_COMMIT_BRANCH == "develop"'

deploy_ansible:
  stage: deploy
  needs: ["docker_push"]
  script:
    - ansible --version
    - ansible -i ansible/inventory.ini deploy -m ping
^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-
```

```
deploy_ansible:
  stage: deploy
  needs: ["docker_push"]
  script:
    - ansible --version
    - ansible -i ansible/inventory.ini deploy -m ping
    - ansible-playbook -i ansible/inventory.ini ansible/deploy.yml
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'

^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Loc
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go
```

YAML validity proof:

The following command was run to verify that the syntax of the *.gitlab-ci.yml* file is correct. The output shows YAML OK, which means that the *.gitlab-ci.yml* file follows valid YAML syntax. This proves that there are no syntax errors in the pipeline configuration and that the GitLab CI/CD runner can parse the file correctly.

```
hasib@runner-24:~/24-repo$ python3 -c "import yaml; yaml.safe_load(open('.gitlab-ci.yml')); print('YAML OK')"
YAML OK
```

Commit and Push:

When running the git commit command, the location was in the develop branch and there were no new changes in the working directory or staging area at that moment.

```
hasib@runner-24:~/24-repo$ git add .gitlab-ci.yml ansible/inventory.ini ansible/deploy.yml
```

```
hasib@runner-24:~/24-repo$ git commit -m "Task3: polish CI/CD (ansible + multi-branch + cache/artifacts/needs)"
```

None of the files *.gitlab-ci.yml*, *ansible/inventory.ini* or *ansible/deploy.yml* were modified or staged. That's why Git naturally showed nothing to commit, working tree clean. This is not an

error, but rather it proves that the working tree was completely clean and there was nothing new to commit.

```
hasib@runner-24:~/24-repo$ git commit -m "Task3: polish CI/CD (ansible + multi-branch + cache/artifacts/needs)"
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
```

There were no new commits in the master branch when I ran the following command.

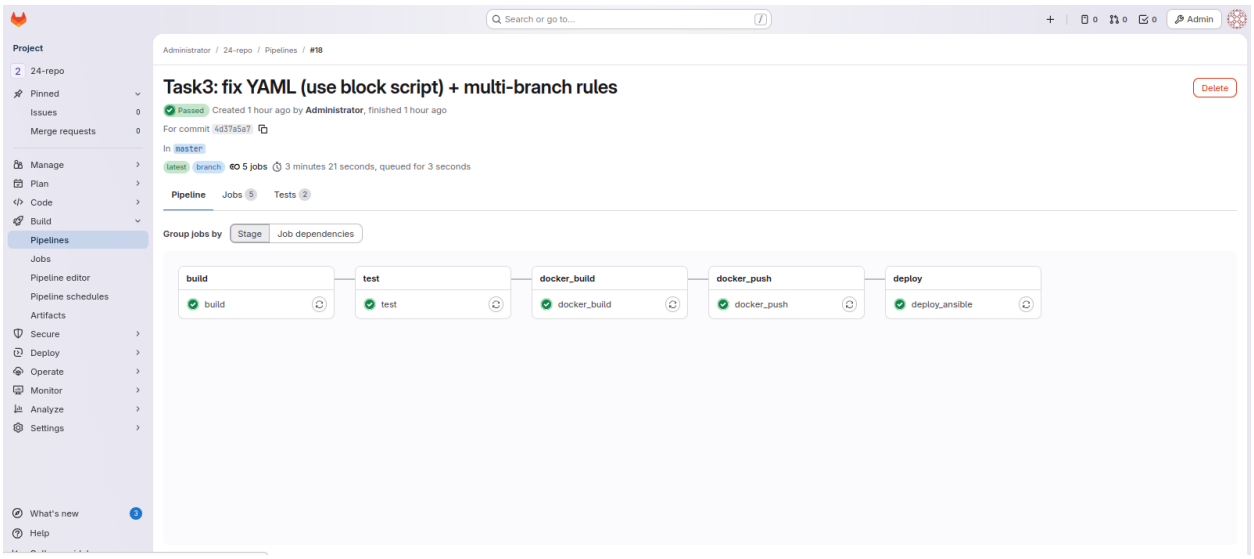
```
hasib@runner-24:~/24-repo$ git push origin master
```

Work was done on the develop branch, but those changes were not committed to master. So, when pushing the master branch, Git Everything was shown as up-to-date because there was nothing new to send to GitLab. This is also not an error, it means that the master branch was unchanged.

```
hasib@runner-24:~/24-repo$ git push origin master
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
Everything up-to-date
```

Pipelines:

According to the *.gitlab-ci.yml* file, the *deploy_ansible* job is only allowed to run on the master branch. So, if the deploy stage runs successfully somewhere, it must be a pipeline triggered from the master branch. But the Git output shows that the position was found on the develop branch and no new commits were pushed to the master push. This means that the deploy pipeline did not run from this develop push.



Pipeline behavior:

- *build* stage ran
- *test* stage ran
- *docker_build* stage ran
- *docker_push* stage ran
- *deploy_ansible* ran

Task 4: Backup and Disaster Recovery Scenario (Bonus)

8. Initiate online/hot backup of your Gitlab server

Solution:

Check if GitLab container is running:

The status of GitLab container on host VM gitlab-24 has been verified by running the following command.

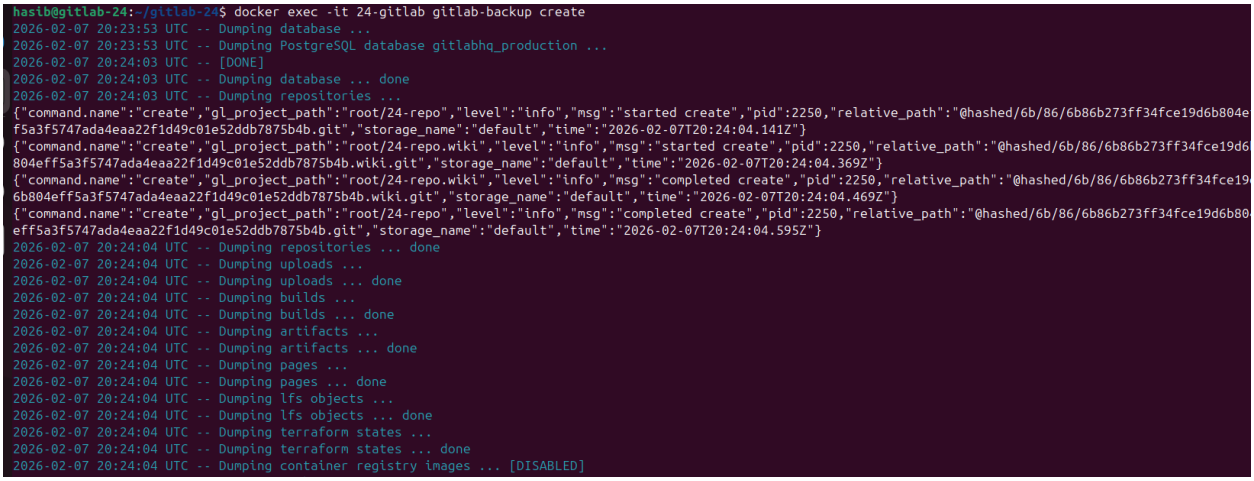
```
hasib@gitlab-24:~/gitlab-24$ docker ps
```

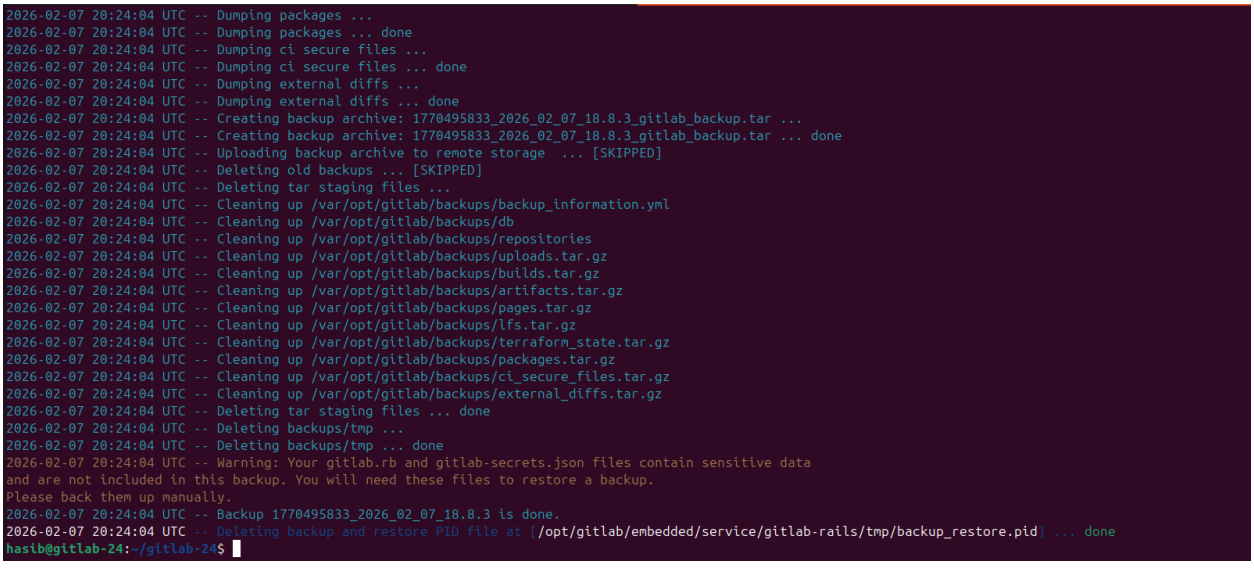
The output showing *24-gitlab gitlab/gitlab-ce:latest* Up 15 minutes (*healthy*) means that the GitLab container is currently running and its internal health check has passed. The *Up* (*healthy*) status confirms that the GitLab services are fully up and running and stable. This verification was very important before taking a hot backup, because the container must be running to run GitLab's *gitlab-backup* create command. If the container is stopped or in an unhealthy state, the backup process will not work or an incomplete backup may be created. This output proves that the system was fully prepared for taking a GitLab backup.

```
hasib@gitlab-24:~/gitlab-24$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS                    PORTS
                                                
```

The output shows Dumping PostgreSQL database *gitlabhq_production ... [DONE],* which means that all GitLab metadata such as users, projects, issues, pipelines have been successfully backed up. Then Dumping *repositories ... completed* create *root/24-repo* shows that all repository data of the 24-repo project has been included in the backup. Uploads, artifacts, packages and CI secure files all show done status, which proves that all important data related to CI/CD has been saved properly. Then Creating backup creates a *.tar* archive and this file is the original GitLab backup.

An important warning is shown that *gitlab.rb* and *gitlab-secrets.json* are not included in this backup. This is because GitLab automatic backup does not include config files. */srv/gitlab-24/config* is kept as a separate bind mount, so these config files can be used separately during restore. *Backup ... is done*. The message shows that the GitLab hot backup has completed successfully and the system is ready for restore or migration.





Verifying Backup Files:

The GitLab backup files are verified by running the following command. Here */var/opt/gitlab/backups* is the default backup directory of GitLab, where the *gitlab-backup* create command saves all the backup archives.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab ls -lh /var/opt/gitlab/backups
```

Multiple *.tar* files in the output mean that multiple backups were taken at different times. The timestamp at the beginning of the file name indicates when the backup was created. Normally, the *.tar* file with the most recent timestamp is used during restore, as it contains the latest data, and that is what happened here. This .tar backup file contains a complete snapshot of GitLab's database, repositories, CI/CD artifacts, etc.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab ls -lh /var/opt/gitlab/backups
total 1.6M
-rw------- 1 git git 770K Feb  7 19:43 1770493403_2026_02_07_18.8.3_gitlab_backup.tar
-rw------- 1 git git 770K Feb  7 20:24 1770495833_2026_02_07_18.8.3_gitlab_backup.tar
```

Confirming if the backup contains the repo:

In this step, the backup file inside the GitLab container has been copied to the host VM and stored in a separate and secure location. The purpose of this is to keep the backup without relying on the container, the same idea is shown here as it is kept in a real environment such as S3, NFS or any other external storage. A dedicated backup directory has been created in the host VM *(~/gitlab-task4-backup)*. This makes it easy to keep lab evidence and do manual verification.

```
hasib@gitlab-24:~/gitlab-24$ sudo mkdir -p ~/gitlab-task4-backup
[sudo] password for hasib:
```

Then, using docker cp, the *gitlab_backup.tar* file, which is the backup file mentioned below in the container, was copied directly to that directory on the host.

```
hasib@gitlab-24:~/gitlab-24$ docker cp 24-gitlab:/var/opt/gitlab/backups/1770493403_2026_02_07_18.8.3_gitlab_backup.tar ~/gitlab-task4-backup/
```

The *docker cp* command safely copied files from the container filesystem to the host filesystem, so the backup was no longer bound to the container.

```
hasib@gitlab-24:~/gitlab-24$ docker cp 24-gitlab:/var/opt/gitlab/backups/1770493403_2026_02_07_18.8.3_gitlab_backup.tar ~/gitlab-task4-backup/
Successfully copied 32.8kB to /home/hasib/gitlab-task4-backup/
```

Verified on the host side by running the following command.

```
hasib@gitlab-24:~/gitlab-24$ ls -lh ~/gitlab-task4-backup
```

The output showed the backup *.tar* file and its size was non-zero, which proved that the data was indeed copied. This backup file was fully prepared for disaster recovery or restore test.

```
hasib@gitlab-24:~/gitlab-24$ ls -lh ~/gitlab-task4-backup
total 772K
-rw------- 1 hasib hasib 770K Feb  7 22:43 1770493403_2026_02_07_18.8.3_gitlab_backup.tar
```

9. Destroy the Gitlab data, and restore the Gitlab from the backup

Solution:

Destroy GitLab data:

The GitLab stack was stopped by running the following command.

```
hasib@gitlab-24:~/gitlab-24$ docker compose down
```

The output showing Container *24-gitlab* Removed means that the container has been stopped and completely removed, and *Network ... Removed* means that the Docker network created for this compose stack has also been removed. The behavior of the down command is to remove the container and network, but the host bind-mounted volume or directory is not automatically deleted. The original GitLab data still remains in the host's bind mount folders.

```
hasib@gitlab-24:~/gitlab-24$ docker compose down
[+] down 2/2
 ✔ Container 24-gitlab       Removed                                                
```

```
hasib@gitlab-24:~/gitlab-24$ docker compose up -d
[+] up 2/2
 ✔ Network gitlab-24_default Created                                                                                0.1s
 ✔ Container 24-gitlab       Created                                                                                0.2s
```

Health check:

The health status of the newly launched GitLab container has been verified by running the following command.

```
hasib@gitlab-24:~/gitlab-24$ docker ps
```

The output initially shows health: starting because after deleting the data directory, GitLab treats it as a fresh install and takes time to initialize the database, run migrations, and start various internal services on the first boot.

```
hasib@gitlab-24:~/gitlab-24$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS                            PORTS                                                                                
```

server, which serves UI and API requests, and *sidekiq* is the background job processor, which handles asynchronous tasks such as pipeline jobs, notifications, imports, etc.

The database and filesystem are directly modified during the restore process. If *puma* or *sidekiq* are running at this time, they can read/write on the same data, which poses a risk of conflict or data corruption. Therefore, these services are stopped before starting the restore.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl stop puma
ok: down: puma: 1s, normally up
```

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl stop sidekiq
ok: down: sidekiq: 1s, normally up
```

The output shows *ok: down: puma ...* and *ok: down: sidekiq ...* which means both services were stopped successfully. This confirms that GitLab is now in a safe state and ready to start the core backup restore process.

Run the restore command:

To restore the backup, the following restore command is run using the *BACKUP ID* from the *.tar* file name**.**

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-backup restore BACKUP= 1770493403_2026_02_07_18.8.3
```

In the restore output, the database was restored first, which means that GitLab's users, projects, issues, pipelines, etc. metadata were successfully restored. Then, in the repositories restore step, the completed restore *root/24-repo* was displayed, which means that the *root/24-repo* repository was restored correctly. Then, uploads, artifacts, packages, and other CI/CD related objects were also displayed in the done status, meaning that all critical data was completely recovered. During the restore, when the *Do you want to continue (yes/no)?* prompt appeared, yes was given. This means that GitLab regenerated the internal *SSH authorized_keys* files, which was necessary to complete the restore. Restore task is done. The message was displayed, which means that the core recovery process was successfully completed.

```
2026-02-07 20:10:38 UTC -- Restoring terraform states ...
2026-02-07 20:10:38 UTC -- Restoring terraform states ... done
2026-02-07 20:10:38 UTC -- Restoring packages ...
2026-02-07 20:10:38 UTC -- Restoring packages ... done
2026-02-07 20:10:38 UTC -- Restoring ci secure files ...
2026-02-07 20:10:38 UTC -- Restoring ci secure files ... done
2026-02-07 20:10:38 UTC -- Restoring external diffs ...
2026-02-07 20:10:38 UTC -- Restoring external diffs ... done
This task will now rebuild the authorized_keys file.
You will lose any data stored in the authorized_keys file.
Do you want to continue (yes/no)? yes

2026-02-07 20:10:47 UTC -- Deleting tar staging files ...
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/backup_information.yml
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/db
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/repositories
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/uploads.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/builds.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/artifacts.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/pages.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/lfs.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/terraform_state.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/packages.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/ci_secure_files.tar.gz
2026-02-07 20:10:47 UTC -- Cleaning up /var/opt/gitlab/backups/external_diffs.tar.gz
2026-02-07 20:10:47 UTC -- Deleting tar staging files ... done
2026-02-07 20:10:47 UTC -- Deleting backups/tmp ...
2026-02-07 20:10:47 UTC -- Deleting backups/tmp ... done
2026-02-07 20:10:47 UTC -- Warning: Your gitlab.rb and gitlab-secrets.json files contain sensitive data
and are not included in this backup. You will need to restore these files manually.
2026-02-07 20:10:47 UTC -- Restore task is done.
2026-02-07 20:10:47 UTC -- Deleting backup and restore PID file at [/opt/gitlab/embedded/service/gitlab-rails/tmp/backup_restore.pid] ... done
```

Reconfigure and restart:

After the restore is complete, GitLab is reconfigured and restarted to get it back into a properly operational state. The following command is used to reapply the GitLab Omnibus configuration, correcting permissions, syncing service configurations, and fixing any inconsistencies that may have resulted from the restore. This step is strongly recommended after the restore, as it allows GitLab to self-heal to the newly restored state.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl reconfigure
```

Seeing *gitlab Reconfigured!* at the end of the output means the reconfiguration was completed successfully.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl reconfigure
[2026-02-07T20:11:07+00:00] INFO: Started Cinc Zero at chefzero://localhost:1 with repository at /opt/gitlab/embedded (One version per cookbook)
Cinc Client, version 18.3.0
Patents: https://www.chef.io/patents
Infra Phase starting
[2026-02-07T20:11:07+00:00] INFO: *** Cinc Client 18.3.0 ***
[2026-02-07T20:11:07+00:00] INFO: Platform: x86_64-linux
[2026-02-07T20:11:07+00:00] INFO: Cinc-client pid: 1042
/opt/gitlab/embedded/lib/ruby/gems/3.2.0/gems/ffi-yajl-2.6.0/lib/ffi_yajl/encoder.rb:42: warning: undefining the allocator of T_DATA class FFI_Yajl::Ext::Encoder::
YajlGen
[2026-02-07T20:11:08+00:00] INFO: Setting the run_list to ["recipe[gitlab-base]"] from CLI options
[2026-02-07T20:11:08+00:00] INFO: Run List is [recipe[gitlab-base]]
[2026-02-07T20:11:08+00:00] INFO: Run List expands to [gitlab-base]
[2026-02-07T20:11:08+00:00] INFO: Starting Cinc Client Run for 24.sne.com
[2026-02-07T20:11:08+00:00] INFO: Running start handlers
[2026-02-07T20:11:08+00:00] INFO: Start handlers complete.
Resolving cookbooks for run list: ["gitlab-base"]
[2026-02-07T20:11:09+00:00] INFO: Loading cookbooks [gitlab-base@0.1.0, package@0.1.0, gitlab@0.0.1, runit@5.1.7, logrotate@0.1.0, postgresql@0.1.0, redis@0.1.0, m
onitoring@0.1.0, registry@0.1.0, mattermost@0.1.0, consul@0.1.0, gitaly@0.1.0, praefect@0.1.0, gitlab-kas@0.1.0, gitlab-pages@0.1.0, letsencrypt@0.1.0, nginx@0.1.0
, acme@4.2.2, crond@0.1.0]
Synchronizing cookbooks:
  - gitlab-base (0.1.0)
  - package (0.1.0)
  - gitlab (0.0.1)
  - runit (5.1.7)
  - logrotate (0.1.0)
  - postgresql (0.1.0)
  - redis (0.1.0)
  - registry (0.1.0)
  - mattermost (0.1.0)
  - monitoring (0.1.0)
  - consul (0.1.0)
```

```
    * link[/opt/gitlab/init/postgres-exporter] action create (up to date)
    * file[/opt/gitlab/sv/postgres-exporter/down] action nothing (skipped due to action :nothing)
    * directory[/opt/gitlab/service] action create (up to date)
    * link[/opt/gitlab/service/postgres-exporter] action create (up to date)
    * directory[/opt/gitlab/service/postgres-exporter/supervise] action create (up to date)
    * directory[/opt/gitlab/service/postgres-exporter/log/supervise] action create (up to date)
    * ruby_block[wait for postgres-exporter service socket] action run (skipped due to not_if)
    * file[/var/log/gitlab/postgres-exporter/current] action touch (skipped due to only_if)
     (up to date)
  * template[/var/opt/gitlab/postgres-exporter/queries.yaml] action create (up to date)
  * consul_service[postgres-exporter] action delete
    * file[/var/opt/gitlab/consul/config.d/postgres-exporter-service.json] action delete (up to date)
     (up to date)
Recipe: gitlab::gitlab-backup-cli_disable
  * template[/opt/gitlab/etc/gitlab-backup-context.yml] action delete (up to date)
  * account[GitLab Backup User] action remove (up to date)
  * group[git] action manage (up to date)
  * group[gitlab-psql] action manage (up to date)
  * group[registry] action manage (up to date)
Recipe: gitlab::database_reindexing_disable
  * crond_job[database-reindexing] action delete
    * file[/var/opt/gitlab/crond/database-reindexing] action delete (up to date)
     (up to date)
[2026-02-07T20:11:26+00:00] INFO: Cinc Client Run complete in 17.893398342 seconds

Running handlers:
[2026-02-07T20:11:26+00:00] INFO: Running report handlers
Running handlers complete
[2026-02-07T20:11:26+00:00] INFO: Report handlers complete
Infra Phase complete, 0/843 resources updated in 19 seconds
gitlab Reconfigured!
```

Then all GitLab services were restarted by running the following command.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl restart
```

The output shows *ok: run:* in front of each service, which means that all core services have been started properly in the post-restore state. This confirms that the restore process has completed and GitLab is now fully operational again.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab gitlab-ctl restart
ok: run: alertmanager: (pid 1516) 0s
ok: run: gitaly: (pid 1527) 0s
ok: run: gitlab-exporter: (pid 1549) 0s
ok: run: gitlab-kas: (pid 1557) 1s
ok: run: gitlab-workhorse: (pid 1567) 0s
ok: run: logrotate: (pid 1583) 0s
ok: run: nginx: (pid 1589) 0s
ok: run: postgres-exporter: (pid 1597) 0s
ok: run: postgresql: (pid 1607) 0s
ok: run: prometheus: (pid 1615) 0s
ok: run: puma: (pid 1626) 1s
ok: run: redis: (pid 1631) 0s
ok: run: redis-exporter: (pid 1639) 1s
ok: run: sidekiq: (pid 1647) 0s
ok: run: sshd: (pid 1653) 1s
```

10. Confirm all the repos and files are there

Solution:

Checking the repositories filesystem level:

The following command was run to verify that the repository data was actually restored at the filesystem level inside the container. GitLab stores all its Git repositories on disk in the */var/opt/gitlab/git-data/*repositories directory, and the clearest signature of a valid Git repository is the *.git* directory (bare repository).

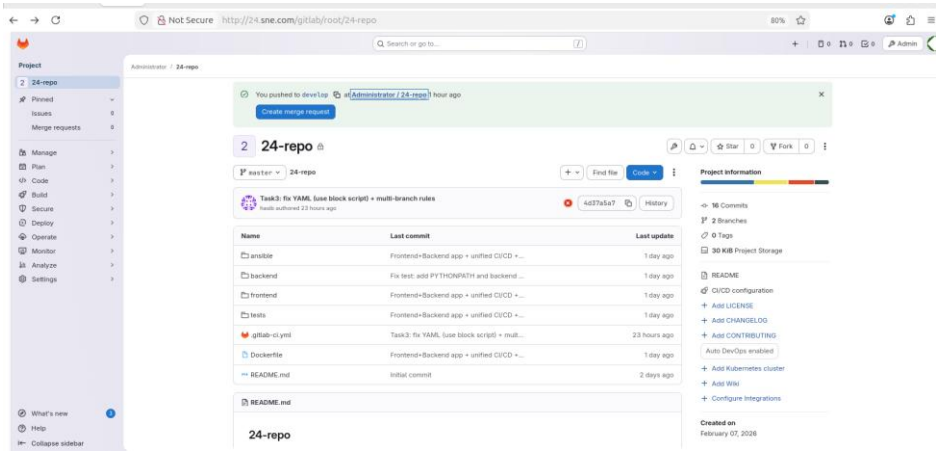
```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab find /var/opt/gitlab/git-data/repositories -name "*.git" | head
```

Here, find is used to search for the *.git* directory and | head is added to limit the output - to provide evidence of existence without showing the entire list. The output shows *@hashed/...git* which means that GitLab's internal hashed storage format is used. The *.wiki.git* shows that not only the main repository but also the associated wiki repository has been restored. This result confirms that the repositories from the backup were successfully restored to disk and that GitLab's repository data is present at the filesystem level.

```
hasib@gitlab-24:~/gitlab-24$ docker exec -it 24-gitlab find /var/opt/gitlab/git-data/repositories -name "*.git" | head
/var/opt/gitlab/git-data/repositories/@hashed/6b/86/6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b.wiki.git
/var/opt/gitlab/git-data/repositories/@hashed/6b/86/6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b.git
/var/opt/gitlab/git-data/repositories/+gitaly/state/@hashed/6b/86/6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b.wiki.git
/var/opt/gitlab/git-data/repositories/+gitaly/state/@hashed/6b/86/6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b.git
```

Web UI Check:

Using the browser, it was visually verified whether the GitLab restore was successful. First, the browser was opened and the GitLab URL *(http://24.sne.com/)* was opened. Then, the credentials that were valid after the restore were used to log in as the root user. After logging in, the *24-repo* project was seen in the Projects list and opened. When the project is accessed from the Repository to the Files section, the files in the repository are properly visible. This directly proves that the *.git* data has been restored to the filesystem and the mapping of the repository to the GitLab database has been recovered properly.



Git clone testing:

The following command was run to verify that the repository data was actually restored at the filesystem level inside the GitLab container. Here, the *.git* directory was searched for with the find command and | head was used to limit the output, but still provide clear evidence of existence.

```
hasib@runner-24:~/24-repo$ git clone http://24.sne.com/gitlab/root/24-repo.git
```

Seeing the *@hashed/...git* format in the output means that GitLab is using its internal hashed storage layout, which is completely normal and production behavior. The presence of the *.git* directory confirms that the repository data was successfully restored to disk from the backup. The presence of *.wiki.git* also provides additional evidence that not only the main repository but also the associated wiki repository was restored. This confirms that GitLab's repository data is present correctly at the filesystem level and that the most important part of the restore process was successful.

```
hasib@runner-24:~/24-repo$ git clone http://24.sne.com/gitlab/root/24-repo.git
Cloning into '24-repo'...
Username for 'http://24.sne.com': root
Password for 'http://root@24.sne.com':
remote: Enumerating objects: 66, done.
remote: Counting objects: 100% (66/66), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 66 (delta 28), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (66/66), 11.07 KiB | 209.00 KiB/s, done.
Resolving deltas: 100% (28/28), done.
```

References

[1] GitLab Inc., "Install GitLab in a Docker container," *GitLab Docs*, 2026. Available: https://docs.gitlab.com/install/docker/installation/

[2] GitLab Inc., "Configure GitLab running in a Docker container," *GitLab Docs*, 2026. Available: https://docs.gitlab.com/install/docker/configuration/

[3] GitLab Inc., "Install GitLab Runner," *GitLab Docs*, 2026. Available: https://docs.gitlab.com/runner/install/

[4] GitLab Inc., "Install GitLab Runner using the official GitLab repositories (Debian/Ubuntu)," *GitLab Docs*, 2026. [Online]. Available: https://docs.gitlab.com/runner/install/linux-repository/

[5] Docker Inc., "Install Docker Engine on Ubuntu," *Docker Documentation*, 2026. Available: https://docs.docker.com/engine/install/ubuntu/

[6] Docker Inc., "Install the Docker Compose plugin (Linux)," *Docker Documentation*, 2026. Available: https://docs.docker.com/compose/install/linux/

[7] GitLab Inc., "Back up GitLab," *GitLab Docs*, 2026. Available: https://docs.gitlab.com/administration/backup_restore/backup_gitlab/

[8] GitLab Inc., "Restore GitLab," *GitLab Docs*, 2026. Available: https://docs.gitlab.com/administration/backup_restore/restore_gitlab/

[9] Ansible (Red Hat), “Inventory basics: formats, hosts, and groups,” *Ansible Documentation*, 2026. Available: https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html

[10] Ansible (Red Hat), “ansible.builtin.ini inventory (INI inventory plugin),” *Ansible Documentation*, 2026. Available: https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/ini_inventory.html

[11] Ansible (Red Hat), “ansible.builtin.env lookup (Read environment variables),” *Ansible Documentation*, 2026. Available: https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/env_lookup.html

[12] Kirill Simonov *et al*., “PyYAML Documentation (yaml.safe_load),” *PyYAML*, 2026. Available: https://pyyaml.org/wiki/PyYAMLDocumentation